



UNIVERSITÀ POLITECNICA DELLE MARCHE

Facoltà di Ingegneria
Corso di Laurea in Ingegneria Informatica e dell'Automazione

Relazione Progetto Programmazione Mobile

Applicazione HotSlice in Kotlin-Flutter per la gestione di una pizzeria

Professore

Prof. Emanuele Storti

Autori

Filippo Montagnoli
Giovanni Prati
Michele Vigliotta

Anno Accademico 2023-2024

Indice

1 Introduzione	4
2 Progettazione e sviluppo	5
2.1 Glossario dei termini	5
2.2 Requisiti	5
2.2.1 Requisiti funzionali	5
2.2.2 Requisiti non funzionali	7
2.3 Casi d'uso	8
2.4 Architettura	12
2.4.1 Kotlin	12
2.4.2 Flutter	16
2.5 Database	17
2.6 Mockup	18
2.7 Approccio e Problematiche di sviluppo	26
2.7.1 Descrizione componenti(Kotlin)	26
2.7.2 Descrizione componenti(Flutter)	42
2.8 Testing	50
2.8.1 Unit Test	50
2.8.2 Instrumented Test	51
3 Conclusioni	56

1 Introduzione

HotSlice è un'applicazione che nasce per facilitare la gestione degli ordini in una pizzeria e quindi supportare la comunicazione tra lo staff del locale e i clienti.

L'app offre tre diversi livelli d'utenza: Cliente, Staff e Admin.

Il Cliente può visualizzare il menu e cercare dei prodotti digitando in una barra di ricerca. Una volta trovato il prodotto desiderato, cliccando su di esso, può aprire la pagina dei dettagli, in cui può aggiungere il prodotto al carrello o aggiungerlo alla lista dei preferiti. Oltre al menù diviso in pizze, bibite e dolci, l'app presenta una sezione dedicata alle offerte, dove è possibile visualizzare la lista di offerte.

Anche qui, come in ogni sezione dell'app dedicata ai prodotti, l'utente può cliccare su un prodotto per visualizzarne dettagli come la foto, la descrizione e il prezzo, per poi poterlo aggiungere al carrello e/o ai preferiti.

Dal carrello è possibile modificare le quantità dei prodotti presenti ed effettuare l'ordine scegliendo il tipo di servizio: servizio al tavolo o d'asporto.

Nella sezione dedicata agli ordini è possibile consultare lo storico completo di tutti gli ordini effettuati, con dettagli come data, totale dell'ordine e prodotti ordinati. Inoltre, lo stato dell'ordine è indicato tramite un'icona e un testo che permettono all'utente di verificare se l'ordine è stato accettato, rifiutato o è ancora in fase di approvazione.

Lo Staff può inserire nuove offerte, modificare ed eliminare offerte già esistenti.

Inoltre, può gestire gli ordini degli utenti, accettandoli o rifiutandoli in base alle esigenze. Lo staff visualizza gli ordini, nell'apposita sezione, suddivisi per tipologia, cioè "al tavolo" o "d'asporto".

L'admin ha la possibilità di modificare tutti gli elementi del menù escluse le offerte.

Inoltre, è presente la sezione dei best-seller, dove l'admin può visualizzare attraverso dei grafici a barre verticali i dati di vendita per ciascun prodotto, con la possibilità di scegliere l'intervallo di tempo tra l'ultima settimana, l'ultimo mese o l'ultimo anno. L'applicazione è stata sviluppata prima in Kotlin per Android e in seguito in Flutter cercando di garantire la massima coerenza in termini di interfaccia utente, utilizzo e funzionalità. Nel corso della relazione, presenteremo i diversi capitoli trattando inizialmente gli argomenti in Kotlin e successivamente illustrando gli stessi dettagli in Flutter.

2 Progettazione e sviluppo

2.1 Glossario dei termini

Di seguito sono riportati dei termini utilizzati di frequente, con relativa descrizione.

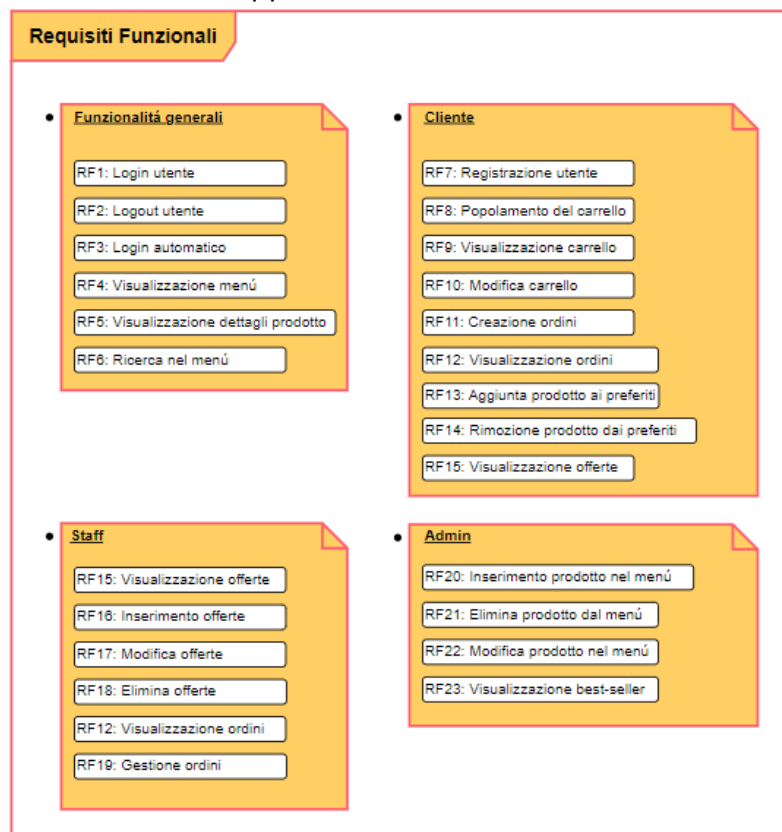
Termini	Descrizione
Best-seller	prodotto più ordinato dai clienti
Bottom navigation bar	Menù di navigazione situato nella parte inferiore dello schermo, che mostra icone disposte orizzontalmente per navigare nell'applicazione.
Search bar	Campo di testo per inserire parole chiave per cercare contenuti all'interno di un'applicazione

2.2 Requisiti

I requisiti dell'applicazione sono suddivisi in requisiti funzionali e non funzionali, come riportato di seguito.

2.2.1 Requisiti funzionali

I requisiti funzionali sono suddivisi in base al ruolo dell'utente per rendere più chiare le funzionalità dell'app.



• Funzionalità generali

Comprendono i requisiti funzionali comuni a tutti i tipi di utente.

- **RF1: Login utente** – L'app permette all'utente di effettuare il login.
- **RF2: Logout utente** – L'app permette all'utente di effettuare il logout.
- **RF3: Login automatico** – L'app permette all'utente di memorizzare le credenziali per effettuare il login in automatico all'avvio.
- **RF4: Visualizzazione menù** – L'app mostra all'utente il menù suddiviso in tre categorie: Pizze, Bibite e Dolci.
- **RF5: Visualizzazione dettagli prodotto** – L'app mostra all'utente i dettagli del prodotto selezionato.
- **RF6: Ricerca nel menù** – L'app consente all'utente di cercare un prodotto nel menù utilizzando una search bar.

• Cliente

Comprendono i requisiti funzionali relativi a utenti di tipo Cliente.

- **RF7: Login utente** – L'app permette all'utente di registrarsi.
- **RF8: Popolamento del carrello** – L'app permette all'utente di aggiungere prodotti al carrello.
- **RF9: Visualizzazione carrello** – L'app mostra all'utente il carrello.
- **RF10: Modifica carrello** – L'app consente all'utente di modificare i prodotti nel carrello.
- **RF11: Creazione ordini** – L'app consente all'utente di creare ordini al tavolo o d'asporto.
- **RF12: Visualizzazione ordini** – L'app mostra all'utente gli ordini e il loro stato.
- **RF13: Aggiunta prodotto ai preferiti** – L'app permette all'utente di aggiungere prodotti alla sua sezione preferiti.
- **RF14: Rimozione prodotto dai preferiti** – L'app consente all'utente di rimuovere prodotti dalla sua sezione preferiti.
- **RF15: Visualizzazione offerte** – L'app mostra all'utente la sezione Offerte.

• Staff

Comprendono i requisiti funzionali relativi a utenti di tipo Staff.

- **RF12: Visualizzazione ordini** – L'app mostra all'utente gli ordini e il loro stato.
- **RF15: Visualizzazione offerte** – L'app mostra all'utente la sezione Offerte.
- **RF16: Inserimento offerte** – L'app consente all'utente di creare offerte visibili nella sezione Offerte.
- **RF17: Modifica offerte** – L'app permette all'utente di modificare le offerte.
- **RF18: Elimina offerte** – L'app consente all'utente di eliminare offerte.
- **RF19: Gestione ordini** – L'app consente all'utente di inserire lo stato dell'ordine (Accettato o Rifiutato) e indicarne l'ora di ritiro se si tratta di un ordine d'asporto.

- Admin

Comprendono i requisiti funzionali relativi a utenti di tipo Admin.

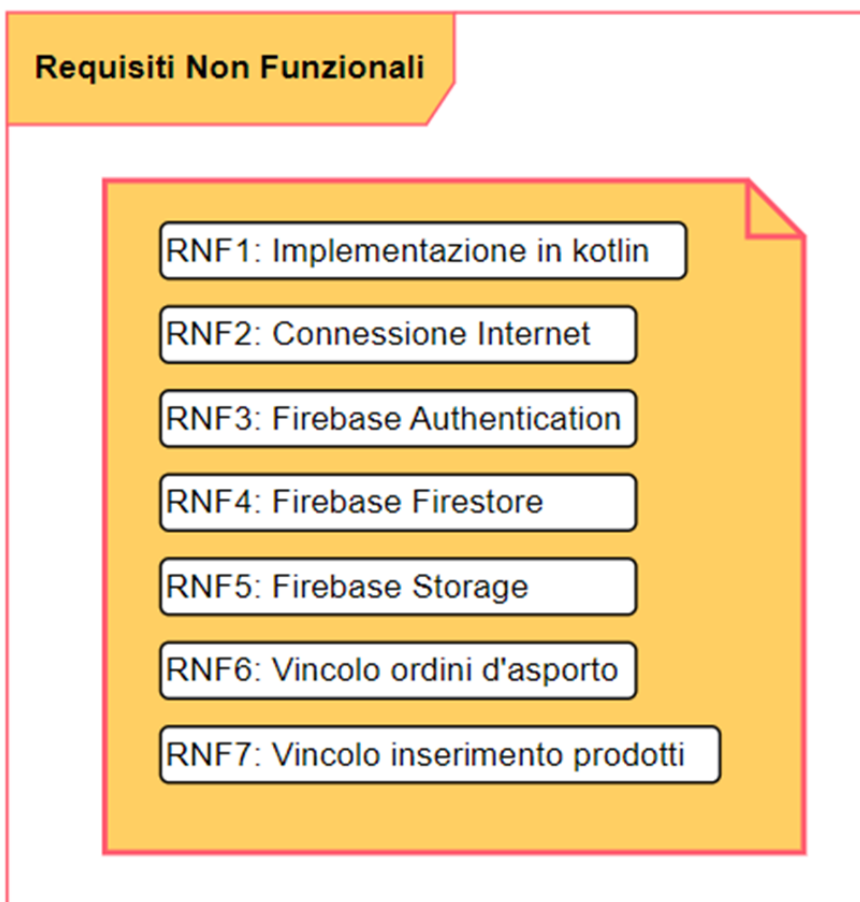
- **RF20: Inserimento prodotto nel menù** – L'app consente all'utente di aggiungere prodotti nel menù.
- **RF21: Elimina prodotto dal menù** – L'app consente all'utente di eliminare prodotti dal menù.
- **RF22: Modifica prodotto nel menù** – L'app consente all'utente di modificare prodotti nel menù.
- **RF23: Visualizzazione best-seller** – L'app mostra all'utente la sezione best-seller contenente statistiche relative ai prodotti più venduti.

- Requisiti funzionali Flutter

Per lo sviluppo in Flutter sono stati seguiti gli stessi requisiti funzionali, ad eccezione dei requisiti **RF13**, **RF14**, **RF20**, **RF21**, **RF22** e **RF23**, che riguardano la gestione dei preferiti e le funzionalità relative agli utenti con ruolo di admin, non presenti in Flutter.

2.2.2 Requisiti non funzionali

I requisiti non funzionali rappresentano i vincoli e le caratteristiche che l'applicazione deve rispettare. Di seguito sono riportati i requisiti non funzionale.



- **RNF1: Implementazione in Kotlin** – L'app deve essere implementata con linguaggio di sviluppo Kotlin.
- **RNF2: Connessione Internet** – L'app deve funzionare solo in presenza di una connessione Internet e impedire all'utente di interagire in assenza di essa.
- **RNF3: Firebase Authentication** – L'app utilizza Firebase Authentication per gestire il login e la registrazione degli utenti.
- **RNF4: Firebase Firestore** – L'app utilizza Firebase Firestore come database per gestire i prodotti (pizze, bibite, dolci, offerte), gli ordini, i preferiti e il ruolo degli utenti.
- **RNF5: Firebase Storage** – L'app utilizza Firebase Storage come database per gestire i prodotti (pizze, bibite, dolci, offerte), gli ordini, i preferiti e il ruolo degli utenti.
- **RNF6: Vincolo ordini d'asporto** – L'app deve limitare l'orario degli ordini d'asporto nell'intervallo di tempo compreso tra le 19 e le 24.
- **RNF7: Vincolo inserimento prodotti** – L'app deve impedire allo staff di inserire offerte già presenti e deve impedire all'admin di inserire prodotti già presenti nel menù.

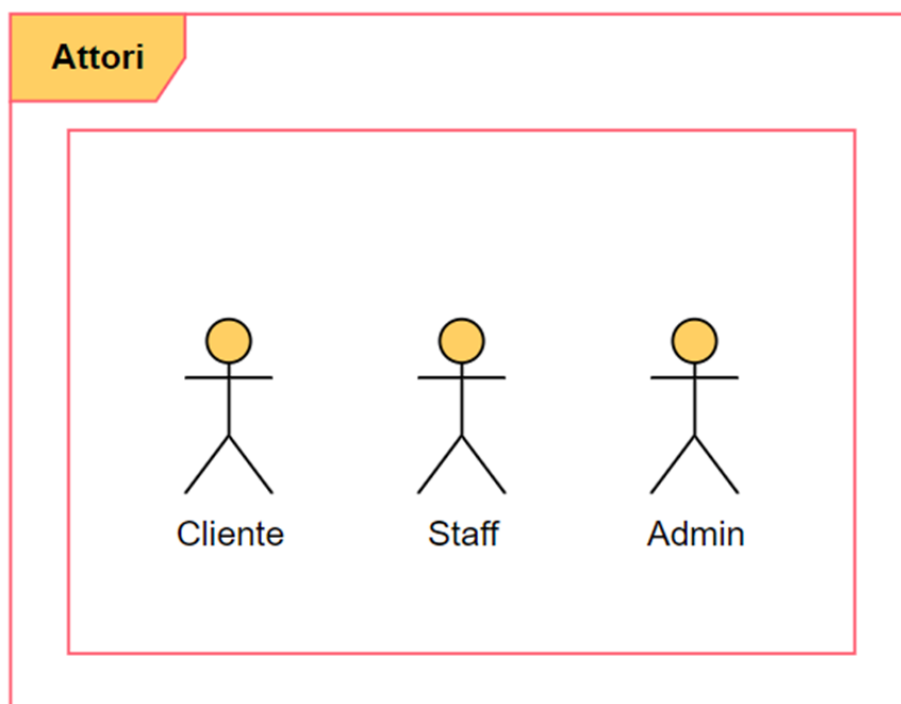
• Requisiti non funzionali Flutter

Per lo sviluppo in Flutter sono stati seguiti gli stessi requisiti non funzionali con due eccezioni. Il vincolo **RNF1** in questo caso ha richiesto lo sviluppo dell'applicazione in Flutter. Mentre il vincolo **RNF7** è stato rispettato solo per l'inserimento delle offerte.

2.3 Casi d'uso

Di seguito sono riportati gli attori che possono interagire con la nostra applicazione e i relativi casi d'uso.

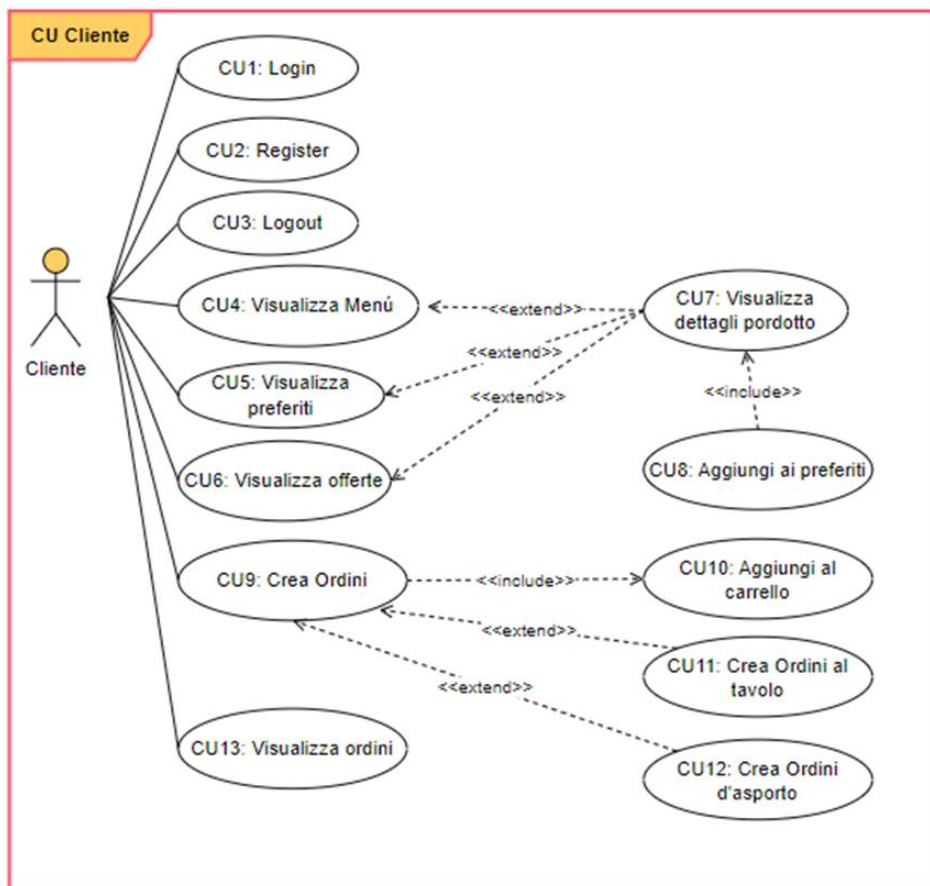
• Attori



• Attori Flutter

In Flutter gli attori sono due, Cliente e Staff.

• Casi d'uso Cliente



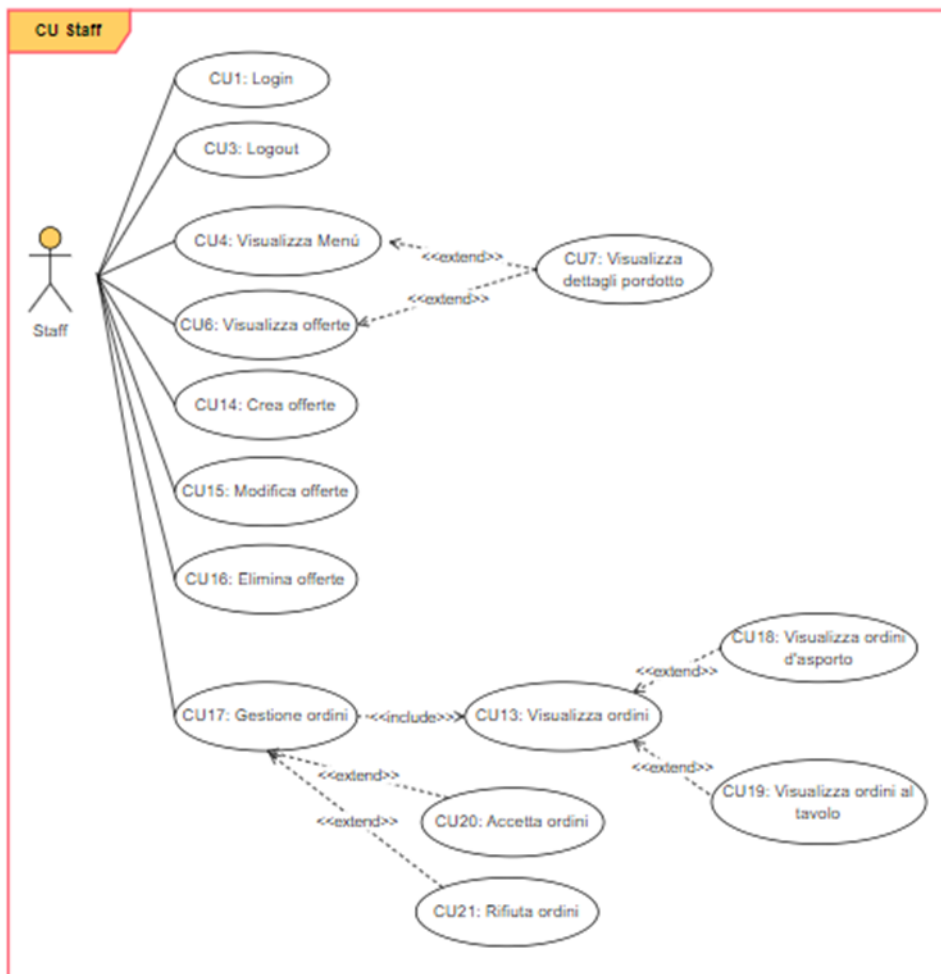
- **CU1: Login** – Il caso d'uso consente all'utente di effettuare il login.
- **CU2: Register** – Il caso d'uso consente all'utente di registrarsi.
- **CU3: Logout** – Il caso d'uso consente all'utente di effettuare il logout.
- **CU4: Visualizza Menù** – Il caso d'uso consente all'utente di visualizzare il menù.
- **CU5: Visualizza Preferiti** – Il caso d'uso consente all'utente di visualizzare i preferiti.
- **CU6: Visualizza Offerte** – Il caso d'uso consente all'utente di visualizzare le offerte.
- **CU7: Visualizza dettagli prodotto** – Il caso d'uso consente all'utente di visualizzare i dettagli del prodotto selezionato. Extend: CU4, CU5, CU6.
- **CU8: Aggiungi ai Preferiti** – Il caso d'uso consente all'utente di aggiungere ai preferiti un prodotto.
Include: CU7.
- **CU9: Crea Ordini** – Il caso d'uso consente all'utente di creare ordini. Include: CU10.
- **CU10: Aggiungi al carrello** – Il caso d'uso consente all'utente di aggiungere prodotti al carrello.
- **CU11: Crea Ordini al tavolo** – Il caso d'uso consente all'utente di creare ordini al tavolo.
Extend: CU9.
- **CU12: Crea Ordini d'asporto** – Il caso d'uso consente all'utente di creare ordini d'asporto.
Extend: CU9.

- **CU13: Visualizza Ordini** – Il caso d’uso consente all’utente di visualizzare gli ordini.

• Casi d’uso Cliente Flutter

I casi d’uso Cliente sono gli stessi esclusi quelli relativi alla gestione dei preferiti.

• Casi d’uso Staff



- **CU1: Login** – Il caso d’uso consente all’utente di effettuare il login.

- **CU3: Logout** – Il caso d’uso consente all’utente di effettuare il logout.

- **CU4: Visualizza Menù** – Il caso d’uso consente all’utente di visualizzare il menù.

- **CU6: Visualizza Offerte** – Il caso d’uso consente all’utente di visualizzare le offerte.

- **CU7: Visualizza dettagli prodotto** – Il caso d’uso consente all’utente di visualizzare i dettagli del prodotto selezionato. Extend: CU4, CU6.

- **CU14: Crea Offerte** – Il caso d’uso consente all’utente di creare nuove offerte.

- **CU15: Modifica Offerte** – Il caso d’uso consente all’utente di modificare offerte esistenti.

- **CU16: Elimina Offerte** – Il caso d’uso consente all’utente di eliminare offerte.

- **CU17: Gestione ordini** – Il caso d’uso consente all’utente di gestire lo stato degli ordini.

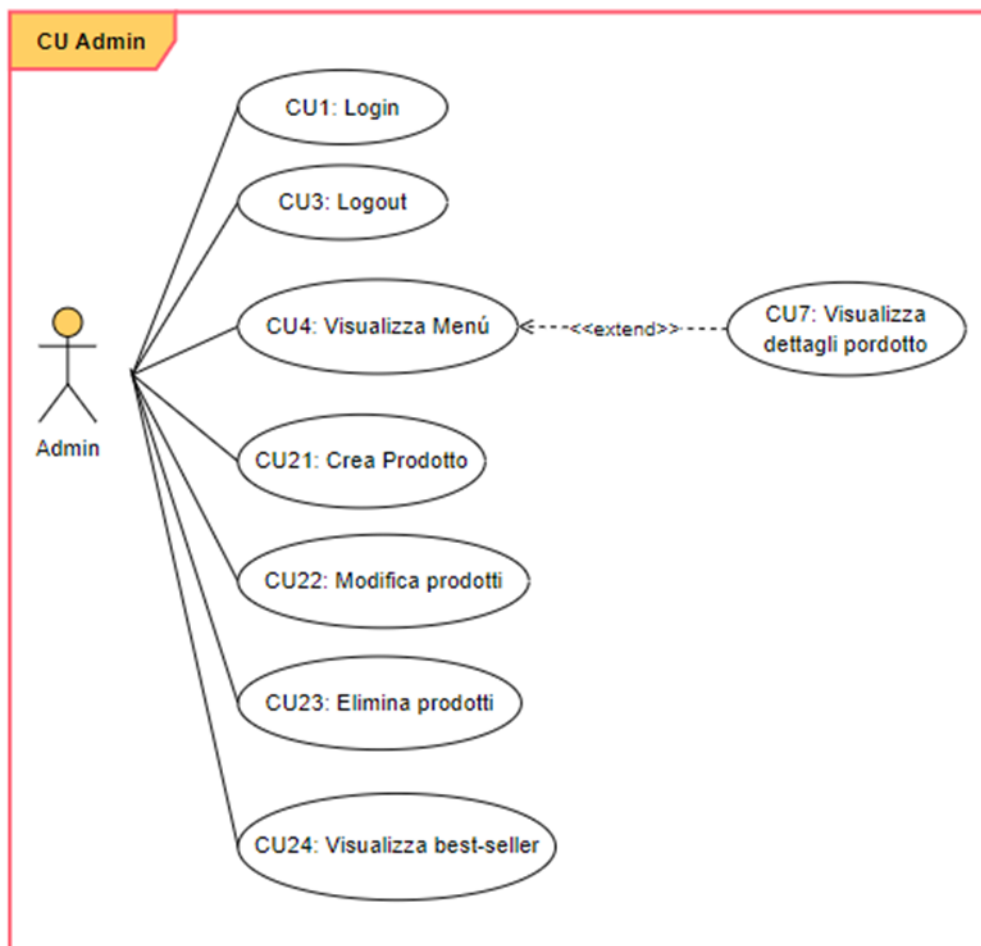
Include: CU13.

- **CU13: Visualizza Ordini** – Il caso d’uso consente all’utente di visualizzare gli ordini.
- **CU18: Visualizza Ordini d’asporto** – Il caso d’uso consente all’utente di visualizzare gli ordini d’asporto. Extend: CU13.
- **CU19: Visualizza Ordini tavolo** – Il caso d’uso consente all’utente di visualizzare gli ordini al tavolo. Extend CU13.
- **CU20: Accetta Ordini** – Il caso d’uso consente all’utente di accettare gli ordini. Extend: CU17.
- **CU20: Rifiuta Ordini** – Il caso d’uso consente all’utente di rifiutare gli ordini. Extend: CU17.

• Casi d’uso Staff Flutter

In questo caso i casi d’uso sono gli stessi.

• Casi d’uso Admin



- **CU1: Login** – Il caso d’uso consente all’utente di effettuare il login.
- **CU3: Logout** – Il caso d’uso consente all’utente di effettuare il logout.
- **CU4: Visualizza Menù** – Il caso d’uso consente all’utente di visualizzare il menù.

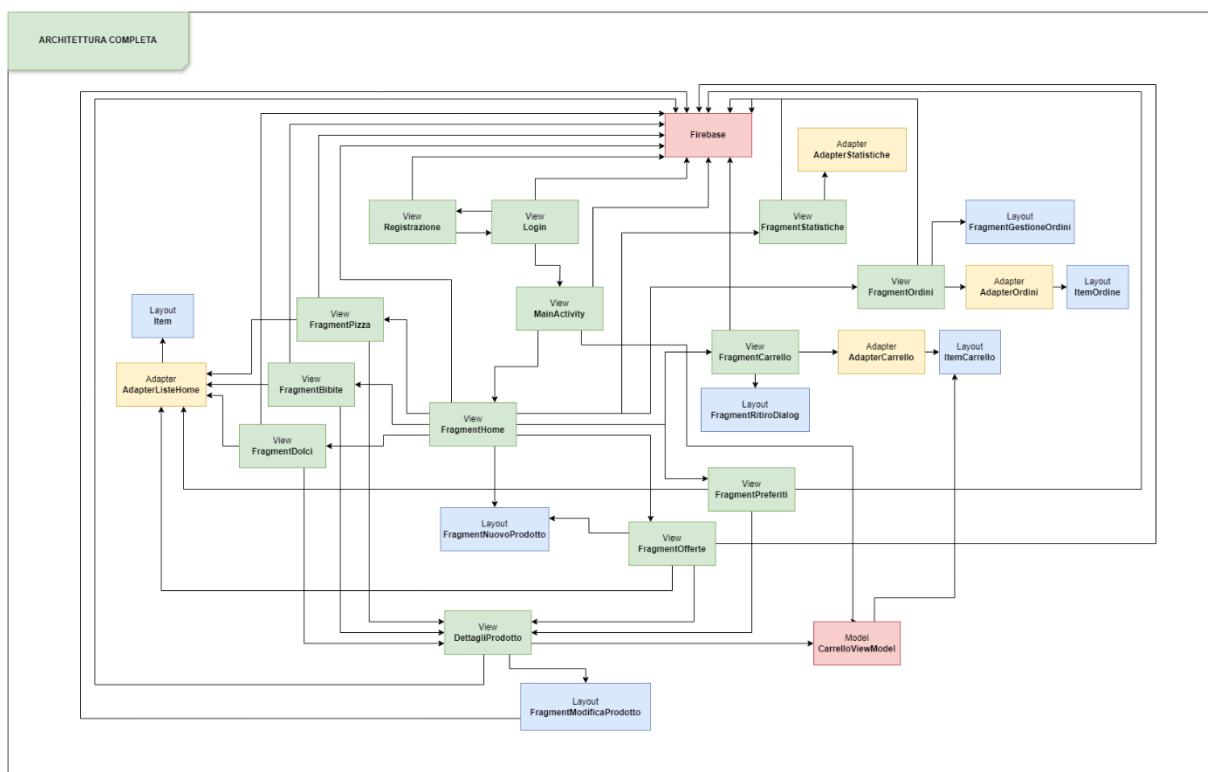
- **CU7: Visualizza dettagli prodotto** – Il caso d’uso consente all’utente di visualizzare i dettagli del prodotto selezionato. Extend: CU4.
- **CU21: Crea Prodotto** – Il caso d’uso consente all’utente di creare un prodotto(pizze/bibite/dolci).
- **CU22: Modifica Prodotto** – Il caso d’uso consente all’utente di modificare prodotti(pizze/bibite/dolci).
- **CU23: Elimina Prodotto** – Il caso d’uso consente all’utente di eliminare prodotti.
- **CU24: Visualizza best-seller** – Il caso d’uso consente all’utente di visualizzare la sezione best-seller.

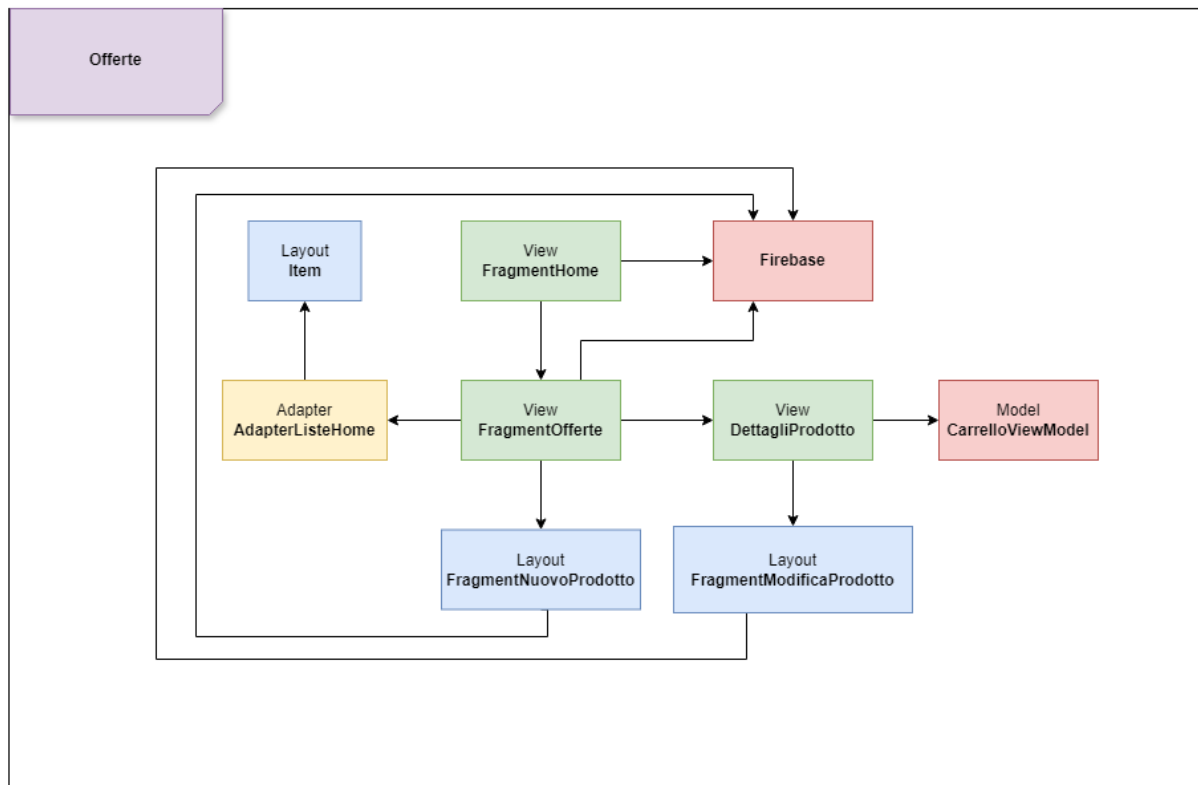
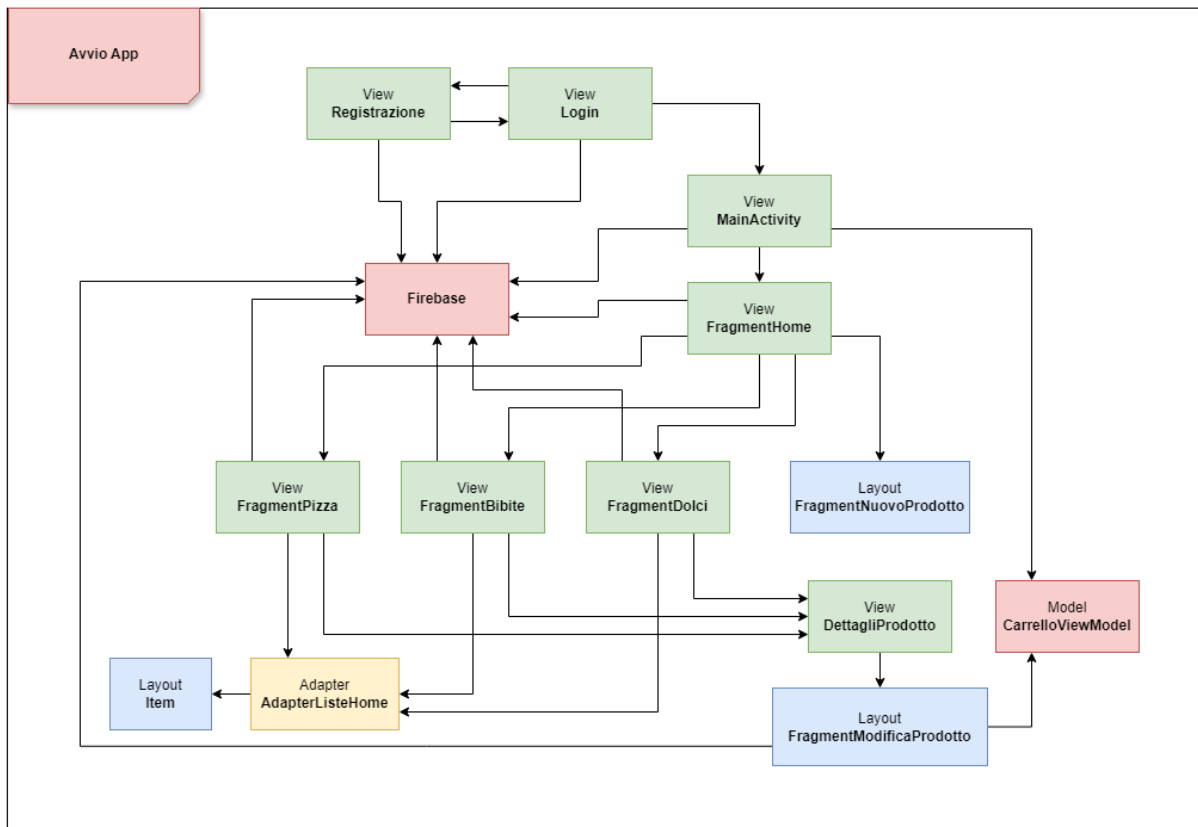
• Casi d’uso Admin Flutter

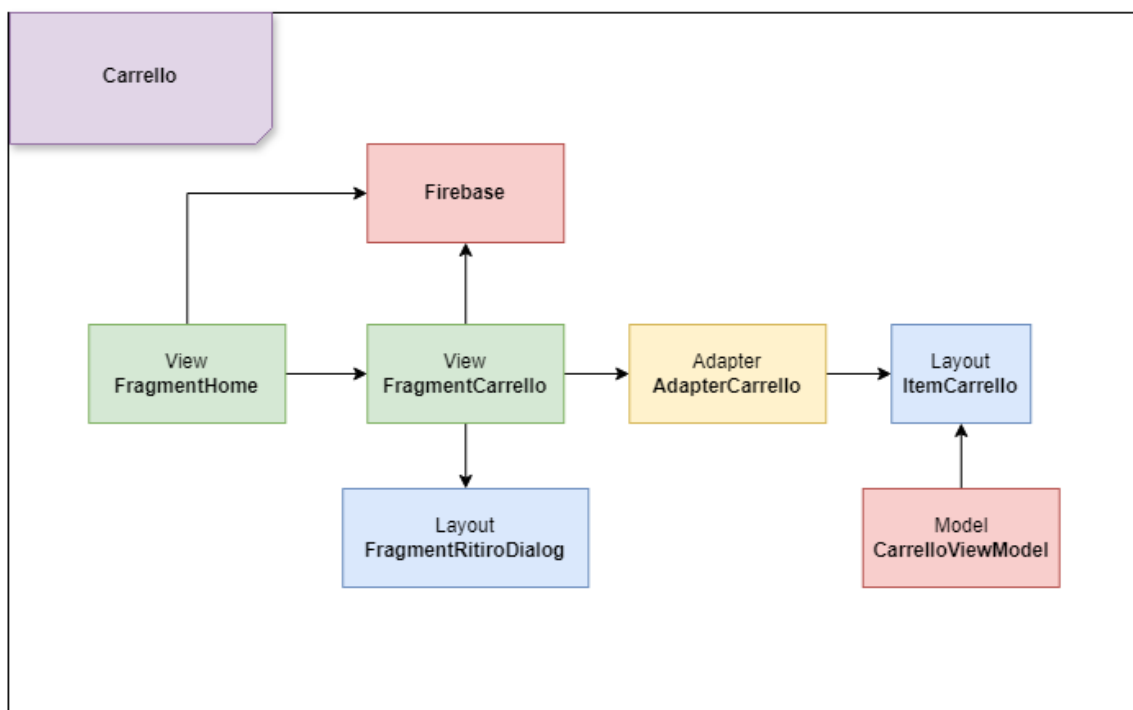
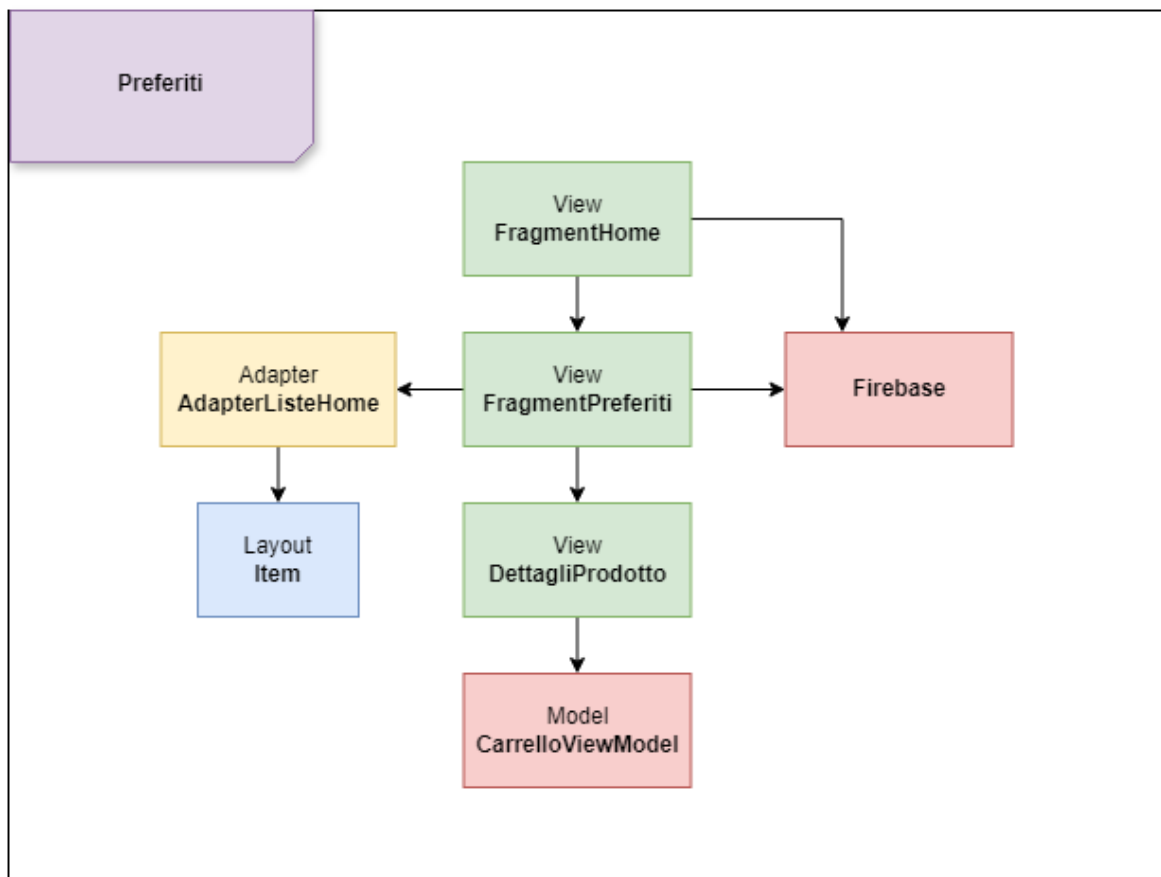
Questi casi d’uso non sono presenti in Flutter.

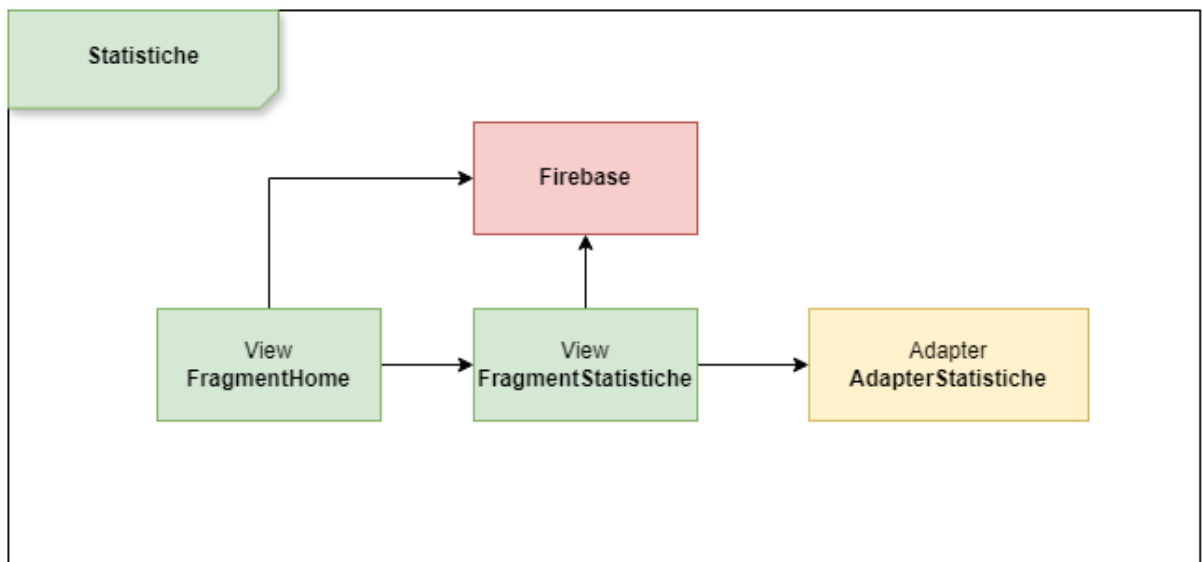
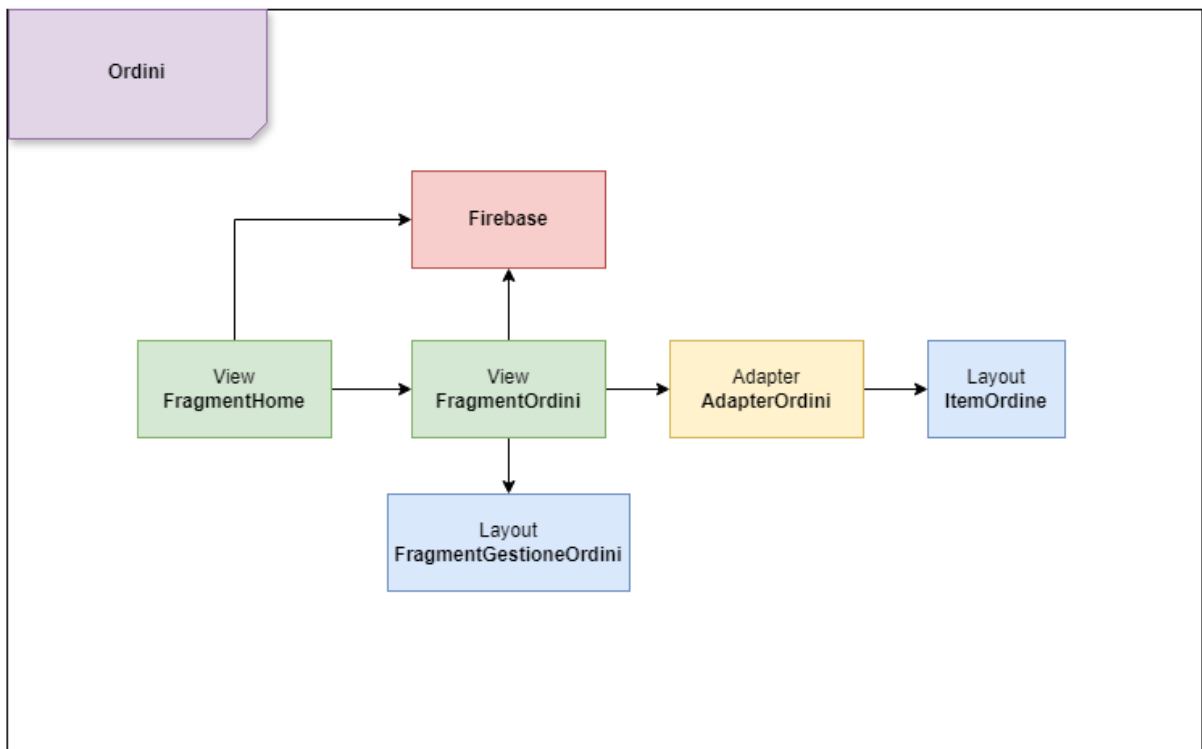
2.4 Architettura

2.4.1 Kotlin

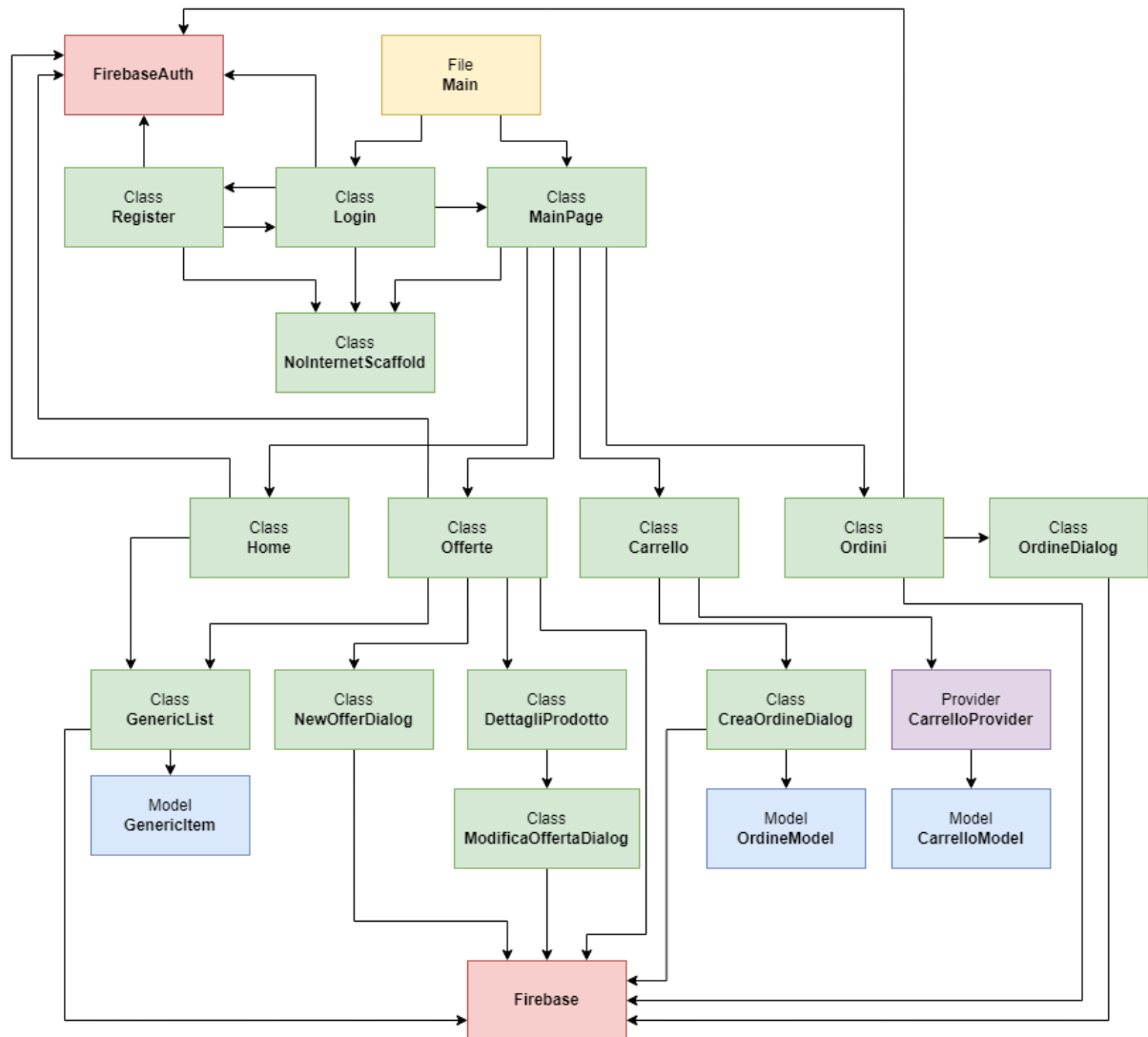








2.4.2 Flutter



2.5 Database

Entrambe le applicazioni Kotlin e Flutter utilizzano Firebase come servizio di database, in particolare abbiamo utilizzato Firebase Authentication per gestire l'autenticazione degli utenti tramite e-mail e password, e Firebase Firestore per strutturare i dati che verranno messi a disposizione dell'applicazione.

Firebase Firestore è un database NoSQL che organizza i dati in raccolte, ogni raccolta può contenere diversi documenti, ogni documento ha una serie di attributi o campi.

Le raccolte 'pizze', 'bibite', 'dolci' e 'offerte' contengono i dettagli degli articoli disponibili nel menu, ogni documento con i seguenti campi: descrizione (string), foto (string), nome (string), prezzo (number).

La raccolta 'preferiti', contenente gli articoli salvati dagli utenti, ha come campi: foto (string), nome (string), prezzo (number) e userId (string), che rappresenta l'identificatore unico dell'utente che ha aggiunto l'elemento ai preferiti.

La raccolta 'ordini' tiene traccia di tutti gli ordini effettuati dagli utenti, i cui dettagli includono: data, ora, nome, descrizione, stato dell'ordine, tipo di ordine, numero di telefono e totale.

La raccolta 'users' contiene documenti che sono gli Id, generati automaticamente, degli utenti. Questi documenti hanno come campo 'role', che rappresenta il ruolo dell'utente.

 (default)	 preferiti  	 5Y1cNV9SogLOxKF2jgrS 
+ Avvia raccolta bibite dolci offerte ordini pizze preferiti > users	+ Aggiungi documento  5Y1cNV9SogLOxKF2jgrS > gB7NnWb1CRltPJI6Srns ursV1y1bCKNrLNLeXIQE	+ Avvia raccolta + Aggiungi campo foto: "acqua_tonica.jpg" nome: "acqua tonica" prezzo: 3 userId: "xn73XS9duUV07zG1J5deBmrzML2"

2.6 Mockup dell'interfaccia e rappresentazione grafica dei casi d'uso

NOTA:(L'applicazione in Flutter implementa solo una parte delle funzionalità di quella in Kotlin, dunque per motivi di completezza e per evitare ridondanze, l'applicazione usata in questa sezione è quella in Kotlin)

Di seguito mostriamo una rappresentazione statica dell'interfaccia utente, con i relativi casi d'uso.

Quando l'applicazione viene lanciata si apre la schermata iniziale ([Foto 1.0](#)) da cui è possibile accedere se si hanno le credenziali, altrimenti si può passare alla pagina di registrazione ([Foto 1.1](#)). Una volta eseguita la registrazione si viene automaticamente rimandati alla pagina di login.

Eseguito l'accesso ci si trova nella homepage ([Foto 1.2](#)). Da questa pagina è possibile fare il logout cliccando sul pulsante in alto a destra, è possibile utilizzare la search bar per cercare un particolare cibo tra la lista presente, oppure cliccando su uno dei tre pulsanti si può scegliere di visualizzare le pizze, le bibite o i dolci. Inoltre, è presente una bottom navigation in basso, da cui ci si può spostare verso le altre pagine dell'applicazione.

Una volta trovato un elemento che vogliamo ordinare è possibile cliccarci per aprire la pagina dei dettagli ([Foto 1.3](#)). In questa pagina visualizziamo tutte le informazioni relative ad un elemento, inclusi prezzo, foto, nome e descrizione. È presente un tasto per aggiungere o togliere l'elemento dai preferiti, è presente un tasto per aumentare il numero di elementi che si vuole ordinare, un tasto per diminuirlo, e il tasto del contatore stesso, in cui si può direttamente digitare un numero. Infine, c'è il tasto per aggiungere gli elementi al carrello, ed un tasto in alto a sinistra per tornare alla pagina precedente.

Se clicchiamo la seconda icona della navigation bar compare la pagina delle offerte ([Foto 1.4](#)). Qui visualizziamo la lista delle offerte, anche qui possiamo selezionare un elemento per vederne i dettagli.

Cliccando la terza icona della navigation bar compare la pagina dei preferiti ([Foto 1.5](#)). In questa pagina visualizziamo la lista degli elementi che abbiamo aggiunto ai preferiti, da cui è possibile cliccare su un elemento per visualizzarne la pagina dei dettagli.

Cliccando la quarta icona della navigation bar compare la pagina del carrello ([Foto 1.6](#)). Qui visualizziamo gli elementi aggiunti al carrello, è possibile aumentare o diminuire la quantità di tali elementi cliccando sui pulsanti gialli con i simboli '+' e '-', premendo il pulsante "Procedi con l'ordine!" si apre un dialog ([Foto 1.7](#)) che ci permette di optare tra il servizio al tavolo o d'asporto, con i relativi campi da compilare in base all'opzione scelta.

Cliccando la quinta icona della navigation bar compare la pagina degli ordini ([Foto 1.8](#)).

Vediamo qui visualizzati tutti i dati relativi a ciascun ordine da noi effettuato.

Queste che abbiamo appena visto sono le interfacce relative ai clienti, vediamo adesso i mockup relativi ad un utente di tipo staff ([Foto 2.0](#)).

Ci troviamo davanti alla stessa pagina home vista in precedenza, cliccando su un elemento della lista si apre la pagina dei dettagli ([Foto 2.1](#)).

Rispetto alla pagina dettagli dei clienti, questa è sprovvista di pulsanti per aggiungere l'ordine al carrello. Cliccando sulla seconda icona della navigation bar compare la pagina delle offerte ([Foto 2.2](#)).

Oltre ad essere presente una lista di prodotti in offerta, è presente anche un pulsante giallo in basso a destra che ci permette di aggiungere nuove offerte, cliccandoci sopra si apre un dialog ([Foto 2.3](#)) dove si possono inserire i dati relativi alla nuova offerta da aggiungere alla lista.

Cliccando invece sulla terza icona della navigation bar compare la pagina degli ordini ([Foto 2.4](#)) dove vengono visualizzati gli ordini al tavolo o d'asporto in base al relativo pulsante che viene cliccato. Sono presenti solo gli ordini effettuati nelle ultime 24 ore, vengono inoltre ordinati dal più recente al più vecchio.

Cliccando su un ordine si apre un dialog ([Foto 2.5](#)) che ci permette di accettare o rifiutare l'ordine in questione.

Infine, vediamo le interfacce relative agli utenti di tipo admin, la pagina home ([Foto 3.0](#)) ha un pulsante giallo in basso a destra, cliccandoci si apre un dialog ([Foto 3.1](#)) per aggiungere un elemento al menù. Premendo su uno degli elementi presenti nella lista si apre la pagina dei dettagli ([Foto 3.2](#))

In basso ci sono due pulsanti, uno apre un dialog per modificare il prodotto ([Foto 3.3](#)) e l'altro per eliminarlo ([Foto 3.4](#)).

Cliccando sulla seconda icona della bottom navigation visualizziamo la pagina delle statistiche ([Foto 3.5](#)). In base all'arco temporale che scegliamo tra le tre opzioni in alto, si può visualizzare la quantità di elementi venduti in quel lasso di tempo.

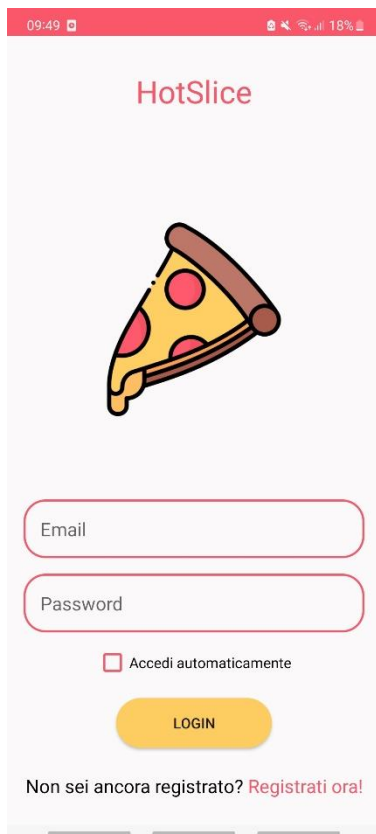


Foto 1.0

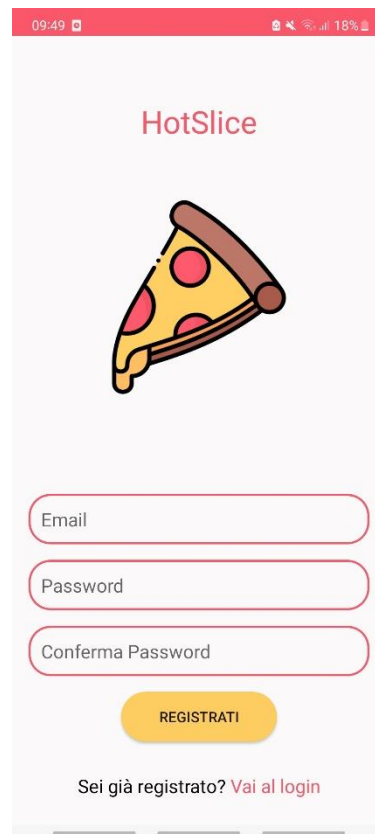


Foto 1.1



Foto 1.2

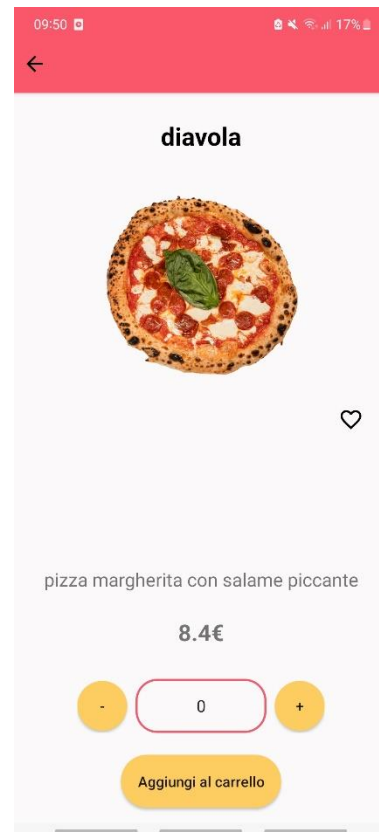


Foto 1.3

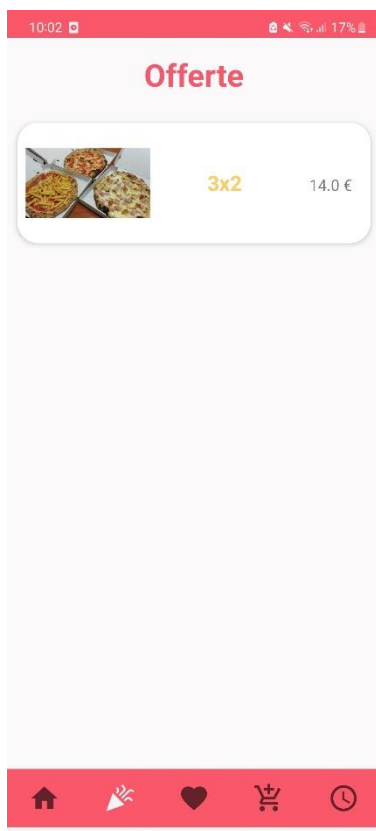


Foto 1.4

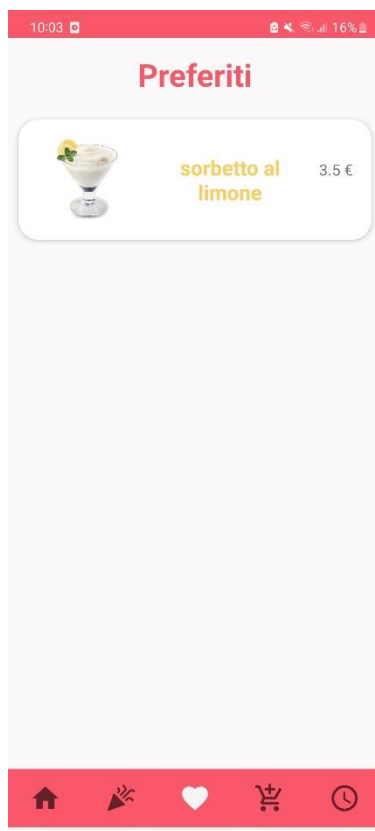


Foto 1.5

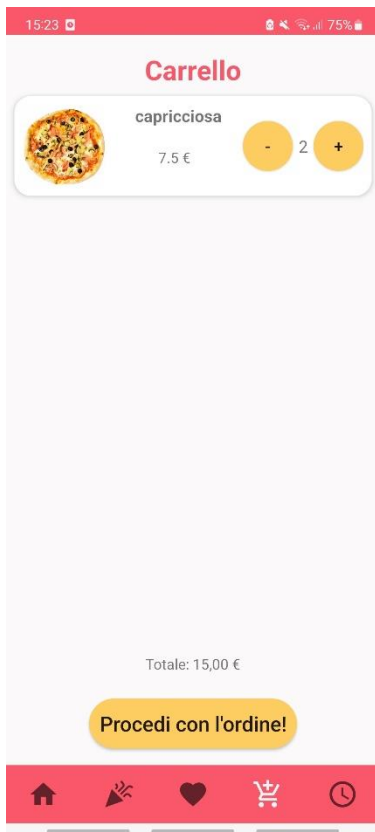


Foto 1.6

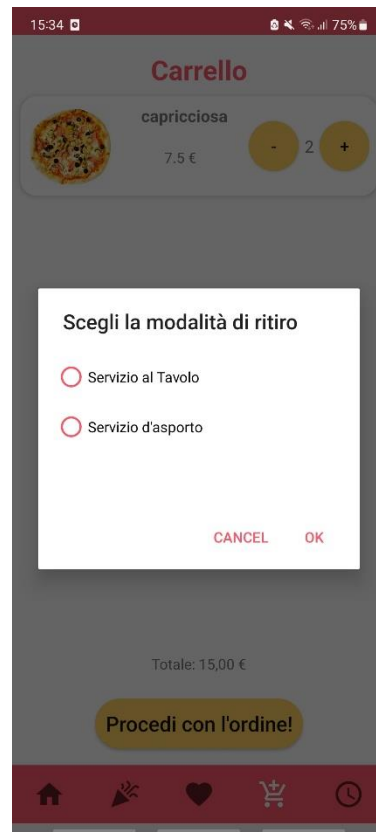


Foto 1.7

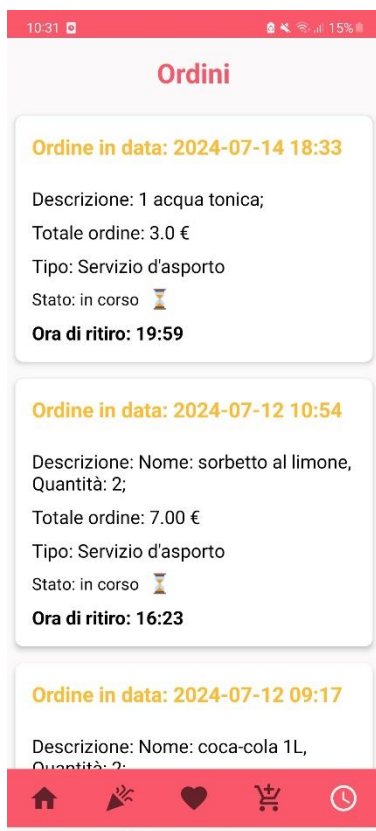


Foto 1.8

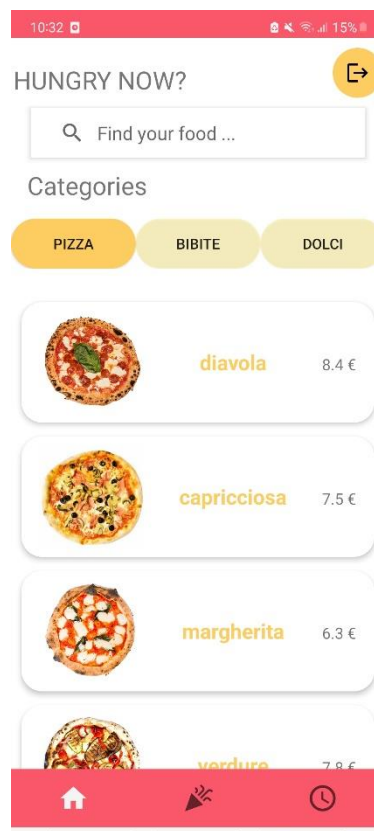


Foto 2.0

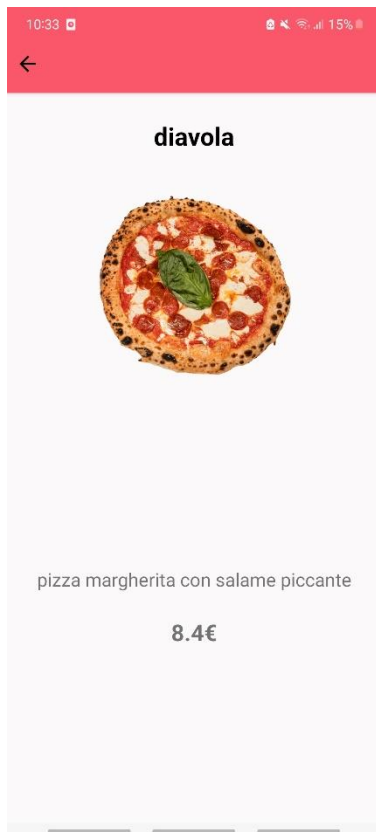


Foto 2.1

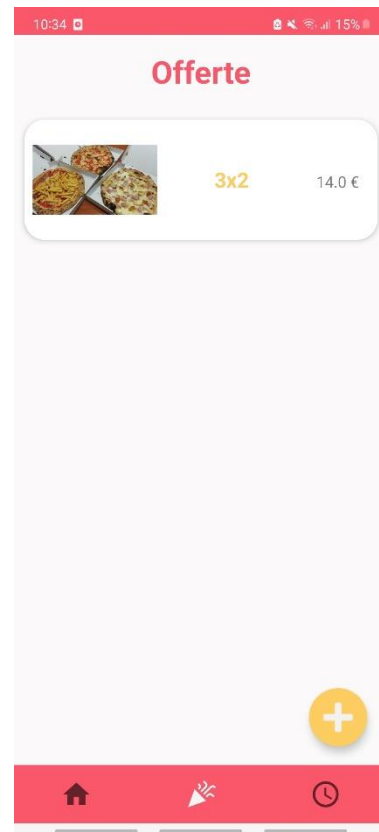


Foto 2.2



Foto 2.3

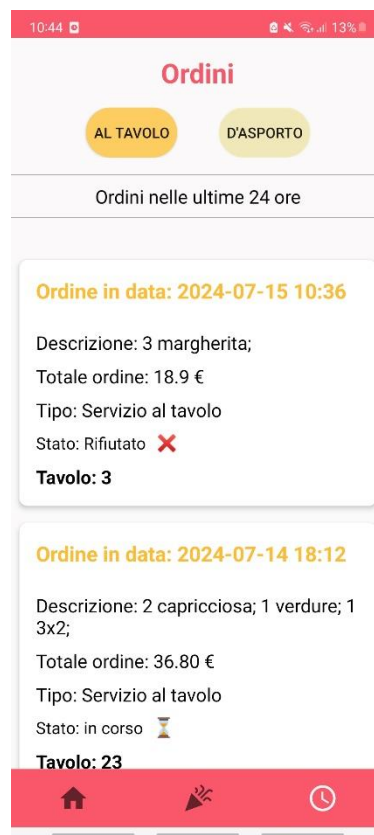


Foto 2.4

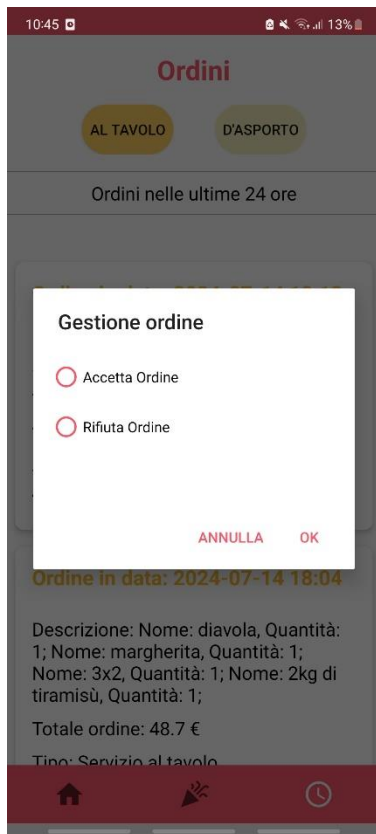


Foto 2.5

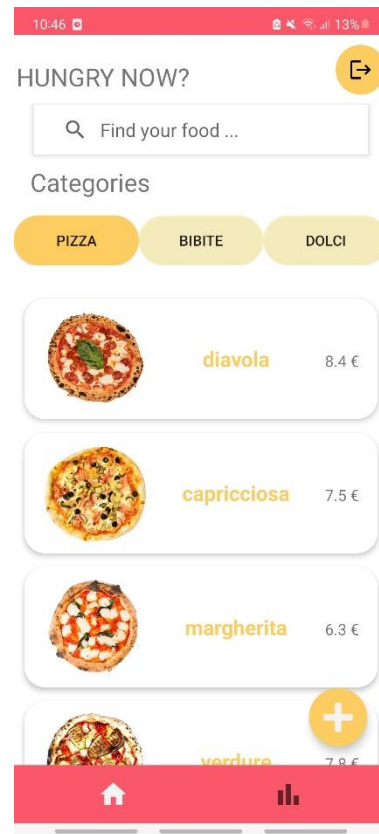


Foto 3.0

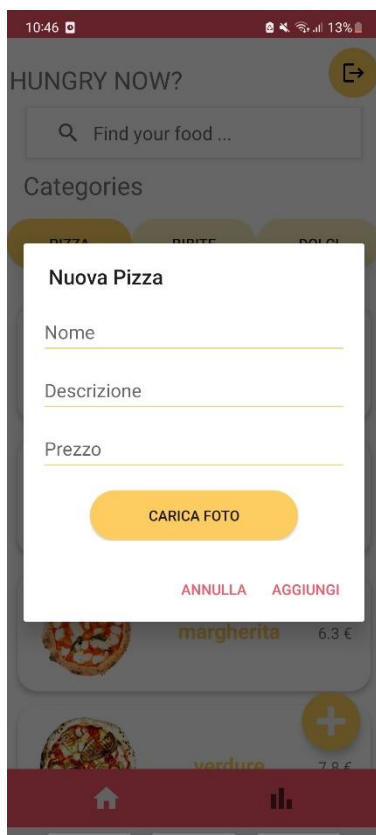


Foto 3.1



Foto 3.2

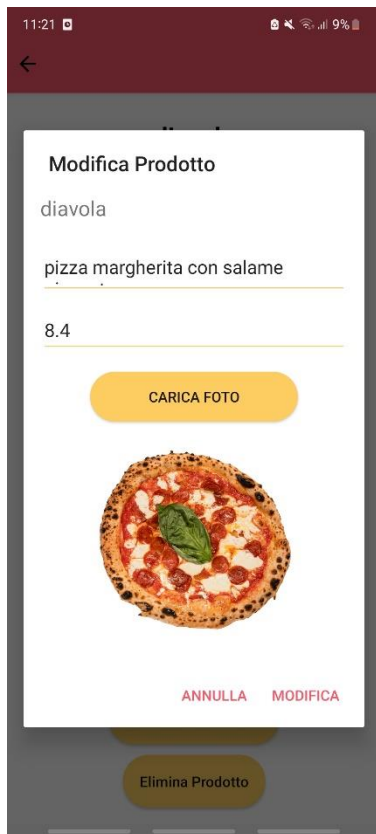


Foto 3.3

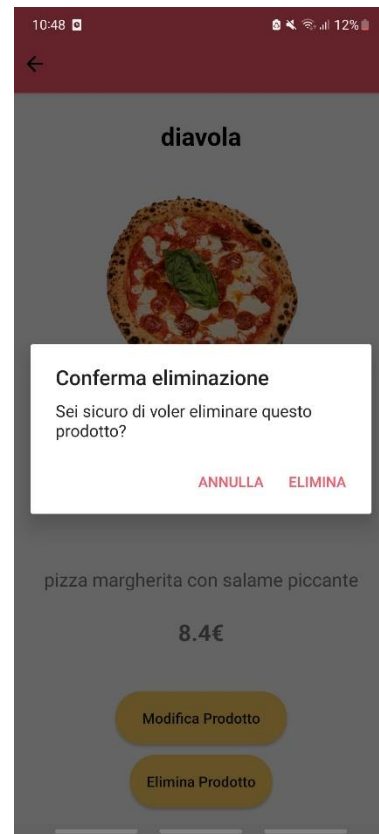


Foto 3.4

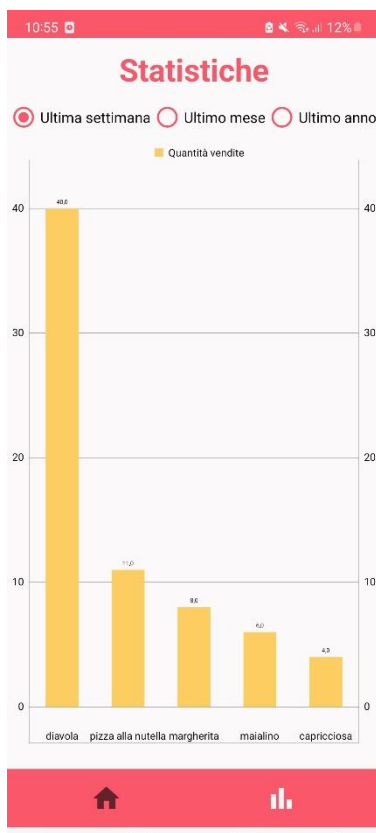


Foto 3.5

2.7 Approccio e Problematiche di Sviluppo

Bug risolti

- Quando veniva modificato un prodotto e si chiudeva il dialog, la pagina dettagli non caricava correttamente le nuove informazioni e mandava in crash l'app
- Quando venivano modificate le informazioni nel database da un utente, un altro utente collegato non vedeva le informazioni aggiornate finché non ricaricava la pagina, ora si aggiorna in contemporanea
- Nella navbar, quando si effettuava il login come admin o staff, prima veniva mostrata la navbar di base e poi veniva corretta, ora si carica direttamente la navbar giusta quando è pronta
- Abbiamo fatto in modo che l'app non possa essere utilizzata in mancanza di connessione, altrimenti togliere la connessione durante l'utilizzo creava inconsistenze.
- Abbiamo fatto in modo che se le immagini non vengono caricate correttamente, l'app mostra un'immagine di default per evitare crash
- Abbiamo avuto problemi nel salvare in modo coerente le informazioni inserite nel database dai due diversi linguaggi di programmazione, abbiamo corretto impostando la formattazione corretta dei dati in modo che fossero caricate allo stesso modo.

Bug

Non abbiamo riscontrato bug che non abbiamo risolto. Abbiamo solo un'inconsistenza tra i due linguaggi di programmazione: in kotlin se un'immagine non viene caricata correttamente viene mostrata una foto di default, mentre in flutter se non viene caricata l'immagine viene mostrato un punto esclamativo rosso di errore.

Librerie di terze parti

Abbiamo utilizzato una libreria per visualizzare le statistiche mettendole in un grafico a colonne: `<com.github.mikephil.charting.charts.BarChart>`

2.7.1 Descrizione Componenti (Kotlin)

MainActivity

```
private var isConnected: Boolean = false
private lateinit var connectivityManager: ConnectivityManager
private lateinit var networkCallback: ConnectivityManager.NetworkCallback
private lateinit var auth: FirebaseAuth
val db = Firebase.firestore
lateinit var role: String
var listaCarrello: ArrayList<ItemCarrello> = arrayListOf()
```

```
private lateinit var binding: ActivityMainBinding
private lateinit var layoutMain: View
private lateinit var layoutNoInternet: View
private lateinit var bottomNavigationView: View
private lateinit var progressBar: ProgressBar
```

onCreate(savedInstanceState: Bundle?): fa l'Inflate della navbar e del layout main facendo partire come fragment di default FragmentHome, e inflate del layout NoInternet. Poi assegna ai tasti della navbar le funzioni per spostarsi tra le pagine. Da qui parte un controllo sul tipo di utente loggato per mostrare solo i tasti della navbar corretti.

checkInternetConnection(): questa funzione effettua un controllo sullo stato di connessione ad internet e fa apparire o no il layout NoInternet in base alla connessione.

RegisterNetworkCallback(): questa funzione serve ad avere un controllo continuo sulla connessione, in modo che l'app non sia utilizzabile senza connessione

OnDestroy() : questa funzione garantisce che quando l'attività o il fragment vengono distrutti, qualsiasi callback di rete registrato venga correttamente deregistrato per evitare perdite di memoria o comportamenti imprevisti.

handleResult(): Questa funzione prende una lista di articoli del carrello (se disponibile), li aggiunge a una lista esistente, unisce gli articoli con lo stesso nome, e registra il risultato.

MergeItemsWithSameName(): prende una lista di oggetti ItemCarrello e restituisce una nuova lista in cui gli oggetti con lo stesso nome (nome) vengono uniti, sommando le loro quantità (quantità)

ReplaceFragment(): Questa funzione serve a sostituire il fragment corrente con il giusto fragment in base al tasto della navbar cliccato

UpdateListaCarrello(): La funzione aggiorna la lista listaCarrello con una nuova lista di oggetti ItemCarrello.

Login

```
private var isInternetConnected: Boolean che indica se c'è connessione internet
private lateinit var connectivityManager: Gestore della connettività per monitorare la rete
private lateinit var networkCallback: Callback per gestire eventi di rete
private lateinit var binding: Binding per accedere alle viste del layout
val db: Riferimento al database Firestore
private lateinit var auth: Riferimento per Firebase Authentication
private lateinit var layoutLogin: Riferimento al layout di login
private lateinit var layoutNoInternet: Riferimento al layout per lo stato senza internet
```

onCreate(savedInstanceState: Bundle?): inizializza Firebase Authentication e il binding del layout, imposta i riferimenti ai layout di login e senza internet, e configura il gestore della connettività registrando il callback di rete e verificando la connessione internet iniziale

checkInternetConnection(): controlla se la connessione internet è disponibile e aggiorna l'interfaccia utente di conseguenza. Se connessi, mostra il layout di login e verifica lo stato di accesso dell'utente utilizzando SharedPreferences. Configura inoltre i listener per il pulsante di login e il testo di registrazione, stilizza il testo di registrazione e gestisce il processo di login e di navigazione verso l'attività principale o di registrazione

registerNetworkCallback(): crea e registra un NetworkCallback per monitorare la disponibilità e la perdita della connessione di rete, chiamando checkInternetConnection() ogni volta che la connessione cambia

onDestroy(): deregistra il networkCallback per evitare perdite di memoria quando l'attività viene distrutta

Register

```
private lateinit var binding: ActivityRegisterBinding
lateinit var firebaseAuth: FirebaseAuth
private var isInternetConnected: Boolean = false
private lateinit var connectivityManager: ConnectivityManager
private lateinit var networkCallback: ConnectivityManager.NetworkCallback
private lateinit var layoutRegister: View
private lateinit var layoutNoInternet: View
```

onCreate(savedInstanceState: Bundle?): Questa funzione fa l'inflate del layout di registrazione utilizzando il binding e inizializza Firebase Authentication. Configura il gestore della connettività per monitorare la disponibilità di rete registrando un callback e verifica la connessione iniziale.

checkInternetConnection(): Controlla lo stato della connessione internet utilizzando il connectivityManager. Se è disponibile una connessione internet attiva, imposta la visibilità del layout di registrazione a View.VISIBLE e nasconde il layout per la mancanza di internet settandolo a View.GONE. Configura quindi un listener sul pulsante di registrazione per gestire il processo di registrazione dell'utente. Verifica le informazioni inserite. Se tutte le condizioni sono soddisfatte, crea un nuovo utente utilizzando Firebase Authentication. Gestisce anche il click sul testo "Already registered?" per reindirizzare l'utente alla schermata di login. Se non c'è connessione internet, nasconde il layout di registrazione e mostra il layout per la mancanza di internet.

registerNetworkCallback(): crea e registra un NetworkCallback per monitorare la disponibilità di rete. Quando cambia lo stato della connessione, chiama checkInternetConnection() per aggiornare l'interfaccia utente.

onDestroy(): questa funzione garantisce che quando l'attività o il fragment vengono distrutti, qualsiasi callback di rete registrato venga correttamente deregistrato per evitare perdite di memoria o comportamenti imprevisti.

FragmentHome

```
private lateinit var auth: FirebaseAuth
private var _binding: FragmentHomeBinding? = null
private val binding get() = _binding!! //getter per _binding (!! restituisce un non null)
private var selectedButton: Button? = null
private lateinit var buttonPizza: Button
private lateinit var buttonBibite: Button
private lateinit var buttonDolci: Button
private lateinit var searchView: SearchView
```

OnCreateView(): questa funzione si occupa di creare e restituire la View principale del Fragment, fa l'inflate utilizzando il binding generato.

onViewCreated(view: View, savedInstanceState: Bundle?): In questa funzione vengono eseguite diverse operazioni per configurare e rendere interattiva l'interfaccia dell'applicazione. Innanzitutto, c'è l'inizializzazione dell'autenticazione tramite Firebase, che permette di gestire l'accesso dell'utente. Poi vengono collegati i pulsanti e la SearchView definiti nel layout tramite il binding, per permettere la navigazione tra le pagine. Poi viene implementato il tasto per il logout che disconnette l'utente dal suo account e lo reindirizza alla schermata di accesso. Infine viene definita la barra di ricerca con una ricerca dinamica.

OnDestroyView(): Questa funzione serve a liberare le risorse legate al binding della vista quando il Fragment viene chiuso, assicurando che la memoria associata al binding venga correttamente eliminata, evitando eventuali perdite di memoria.

selectButton(button: Button): Questa funzione gestisce il cambiamento di stato visivo dei pulsanti.

FragmentPizza

```
private lateinit var recyclerView: RecyclerView
lateinit var pizzaAdapter: AdapterListeHome
var pizzaList = mutableListOf<Item>()
private val REQUEST_CODE_DETAGLI = 100
private val REQUEST_CODE_DETAGLI_PRODOTTO = 200
private lateinit var auth: FirebaseAuth
private lateinit var db : FirebaseFirestore
lateinit var role: String
private lateinit var progressBar: ProgressBar
```

onCreateView(inflater: LayoutInflater, container: ViewGroup?, savedInstanceState: Bundle?): Questa funzione fa l'inflate infla il layout pizza_fragment, che contiene il RecyclerView e altri elementi visivi necessari per visualizzare le pizze. Una volta ottenuta la vista associata al layout, viene preparata l'interfaccia: Viene configurato il RecyclerView per mostrare l'elenco delle pizze, impostando un LayoutManager che definisce come devono essere posizionati gli elementi nella lista. Successivamente, viene assegnato al RecyclerView l'AdapterListeHome e vengono inizializzati i riferimenti al database Firebase e all'autenticazione Firebase. Quando viene premuto il pulsante FloatingActionButton, viene mostrato il dialog FragmentNuovoProdotto. Per garantire che solo gli amministratori possano aggiungere nuove pizze, il codice verifica il ruolo dell'utente loggato. Infine, quando un elemento della lista viene cliccato, il codice prepara un Intent per avviare l'attività DettagliProdottoActivity. A seconda del ruolo dell'utente, viene utilizzato un codice di richiesta diverso per aprire la vista dettagli in modo corretto.

onActivityResult(requestCode: Int, resultCode: Int, data: Intent?): Questa funzione gestisce i risultati restituiti dalle attività figlie e aggiorna di conseguenza l'interfaccia utente.

FetchDataFromFirebase(): Questa funzione aggiorna dinamicamente l'interfaccia utente di un Fragment con i dati delle pizze dal database Firebase. Utilizza un listener per ricevere aggiornamenti in tempo reale dalla collezione "pizze".

filterList(query: String): questa funzione permette agli utenti di cercare e visualizzare solo le pizze il cui nome contiene la stringa di ricerca specificata.

FragmentBibite

```
private lateinit var recyclerView: RecyclerView
private lateinit var bibiteAdapter: AdapterListeHome
private val bibiteList = mutableListOf<Item>()
private val REQUEST_CODE_DETAGLI = 100
private val REQUEST_CODE_DETAGLI_PRODOTTO = 200
private lateinit var auth: FirebaseAuth
val db = Firebase.firestore
lateinit var role: String
private lateinit var progressBar: ProgressBar
```

onCreateView(): configura la vista per il fragment bibite_fragment. Viene eseguito l'inflate sul layout del fragment. Il recyclerView viene configurato con un layout manager lineare e associato a all'AdapterListeHome. Successivamente, viene chiamata la funzione fetchDataFromFirebase per caricare i dati delle bibite da Firebase. Viene eseguito poi un controllo sul ruolo dell'utente loggato. Se l'utente è un admin, il floatingButton viene reso visibile. Infine, viene gestito il click sugli elementi della lista: quando un elemento viene cliccato, viene avviata l'attività DettagliProdottoActivity con i dettagli del prodotto selezionato, adattando il comportamento in base al ruolo dell'utente).

onActivityResult(requestCode: Int, resultCode: Int, data: Intent?): gestisce i risultati delle attività che sono state avviate da questo fragment. Recupera una lista di oggetti ItemCarrello dall'intent di ritorno e chiama la funzione handleResult della MainActivity per gestire questi dati.

fetchDataFromFirebase(): aggiorna in tempo reale la lista di bibite dal database Firestore. Utilizza un addSnapshotListener per controllare le modifiche alla collezione "bibite". Se i dati sono presenti, svuota bibiteList, aggiunge le nuove bibite, aggiorna l'adapter bibiteAdapter, mostra la recyclerView e nasconde la progressBar.

filterList(query: String): filtra la lista di bibite in base alla query della search view, aggiorna l'adapter con la lista filtrata e, se la lista filtrata è vuota, mostra un messaggio toast indicando che la bibita non è presente nel menu.

FragmentDolci

```
private lateinit var recyclerView: RecyclerView
private lateinit var dolciAdapter: AdapterListeHome
private val dolciList = mutableListOf<Item>()
private val REQUEST_CODE_DETAGLI = 100
private val REQUEST_CODE_DETAGLI_PRODOTTO = 200
private lateinit var auth: FirebaseAuth
val db = Firebase.firestore
lateinit var role: String
private lateinit var progressBar: ProgressBar
```

onCreateView(): viene creato e configurato il layout del fragment per visualizzare la lista dolci. Viene inizializzato l'adapterListeHome con la lista di dolci e collegato al recyclerView. Viene avviato il caricamento dei dati da Firebase. Viene creato un pulsante che permette di aggiungere nuovi dolci

visibile solo se l'utente loggato è di tipo admin. Quando un elemento della lista viene cliccato, si apre la DettagliActivity per visualizzare i dettagli del dolce, con permessi differenti per l'amministratore e per altri utenti.

onActivityResult(requestCode: Int, resultCode: Int, data: Intent?): gestisce i risultati delle attività avviate. Quando l'attività di dettaglio restituisce un codice di risultato 200, recupera gli articoli dal carrello e chiama il metodo `handleResult` nell'attività principale per gestire questi articoli. Se il codice di risultato è 201 e la richiesta era per i dettagli del prodotto, mostra un messaggio che indica che il prodotto è stato eliminato. Infine, ricarica i dati da Firebase.

fetchDataFromFirebase(): recupera i dati dalla collezione "dolci" di Firebase in tempo reale, grazie a un `SnapshotListener`. Se ci sono errori, vengono registrati nel log. Se i dati vengono recuperati con successo e non sono vuoti, la lista dei dolci viene aggiornata con i nuovi dati e la vista del `recyclerView` viene resa visibile mentre la `progressBar` viene nascosta.

filterList(query: String): filtra la lista dei dolci in base alla query fornita, ignorando maiuscole e minuscole. Aggiorna l'adapter della lista con i risultati filtrati. Se la lista filtrata è vuota, mostra un messaggio che indica che il dolce non è presente nel menù.

FragmentOfferte

```
private lateinit var recyclerView: RecyclerView
private lateinit var offerteAdapter: AdapterListeHome
private val offerteList = mutableListOf<Item>()
private val REQUEST_CODE_DETAGLI = 100
private val REQUEST_CODE_DETAGLI_PRODOTTO = 200
private lateinit var auth: FirebaseAuth
val db = Firebase.firestore
lateinit var role: String
private lateinit var progressBar: ProgressBar
```

onCreateView(inflater: LayoutInflater, container: ViewGroup?, savedInstanceState: Bundle?): viene eseguito il layout inflating del fragment e viene inizializzata la vista. Viene impostato il layout manager per il `recyclerView` e viene creato un `offerteAdapter` con una lista vuota, che viene assegnato al `recyclerView`. Viene poi chiamato `fetchDataFromFirebase` per caricare i dati da Firebase. Un pulsante flottante permette di aggiungere una nuova offerta, aprendo un dialogo con un bundle contenente informazioni sulla nuova offerta. Viene verificato il ruolo dell'utente corrente tramite `Firebase Auth` e, se il ruolo è "staff", il pulsante flottante viene reso visibile. Inoltre, viene definito un click listener per gli elementi della lista delle offerte, che apre una nuova attività con i dettagli dell'offerta.

onActivityResult(requestCode: Int, resultCode: Int, data: Intent?): Nel metodo `onActivityResult`, vengono gestiti i risultati delle attività avviate. Se la richiesta è per i dettagli dell'offerta e il risultato è 200, vengono recuperati gli articoli del carrello e passati alla `MainActivity` tramite `handleResult`. Se il risultato è 201 e la richiesta è per i dettagli del prodotto, viene mostrato un messaggio che indica che il prodotto è stato eliminato. Infine, viene chiamato `fetchDataFromFirebase` per ricaricare i dati.

fetchDataFromFirebase(): Il metodo `fetchDataFromFirebase` utilizza un `addSnapshotListener` per aggiornare in tempo reale la lista delle offerte dalla collezione "offerte" di Firebase. Se ci sono errori, vengono registrati nel log. Se i dati vengono recuperati con successo e non sono vuoti, la lista delle offerte viene aggiornata e resa visibile, mentre la `progressBar` viene nascosta. Se i dati sono nulli, viene registrato un messaggio di log.

FragmentPreferiti

```
private lateinit var recyclerView: RecyclerView
private lateinit var preferitiAdapter: AdapterListeHome
private val preferitiList = mutableListOf<Item>()
private val REQUEST_CODE_DETAGLI = 100
private lateinit var progressBar: ProgressBar
```

onCreateView(inflater: LayoutInflater, container: ViewGroup?, savedInstanceState:

Bundle?): viene eseguito il layout inflating del fragment "preferiti" e vengono inizializzate le varie componenti della vista, come la progressBar e il recyclerView. Viene impostato un layout manager per il recyclerView e viene creato un preferitiAdapter con una lista inizialmente vuota, che viene poi assegnato al recyclerView. Viene chiamato `fetchDataFromFirebase` per caricare i dati da Firebase. Inoltre, viene definito un click listener per gli elementi della lista dei preferiti, che apre una nuova attività con i dettagli del prodotto selezionato.

onActivityResult(requestCode: Int, resultCode: Int, data: Intent?): vengono gestiti i risultati delle attività avviate. Se la richiesta è per i dettagli dell'articolo preferito e il risultato è 200, vengono recuperati gli articoli del carrello e passati alla MainActivity tramite `handleResult`. Infine, viene chiamato `fetchDataFromFirebase` per ricaricare i dati.

fetchDataFromFirebase(): svuota la lista dei preferiti esistente e recupera i dati dalla collezione "preferiti" di Firebase, filtrando per l'ID dell'utente corrente. Per ogni documento recuperato, viene creato un oggetto `Item` e aggiunto alla lista dei preferiti. L'adapter viene notificato dei cambiamenti e la lista dei preferiti viene aggiornata e resa visibile, mentre la progressBar viene nascosta. In caso di errore nel recupero dei dati, viene registrato un messaggio di log.

FragmentCarrello

```
private lateinit var recyclerView: RecyclerView
private lateinit var adapter: AdapterCarrello
private lateinit var auth: FirebaseAuth
private lateinit var carrello: List<ItemCarrello>
var string: String = ""
val db = FirebaseFirestore.getInstance()
private lateinit var totaleTextView: TextView
private lateinit var progressBar: ProgressBar
```

onCreateView(inflater: LayoutInflater, container: ViewGroup?, savedInstanceState:

Bundle?): il layout del fragment "carrello" viene gonfiato e restituito

onViewCreated(view: View, savedInstanceState: Bundle?): la progressBar e il recyclerView vengono inizializzati, con il recyclerView configurato con un layout manager lineare. Si ottiene la lista del carrello dalla MainActivity e viene impostata la totaleTextView per mostrare il totale dell'ordine. Un adapter per il carrello viene creato e assegnato al recyclerView. Viene aggiornato il totale iniziale del carrello e si imposta un listener per aggiornare la lista del carrello e nascondere la progressBar quando i dati vengono aggiornati. Il pulsante di ordinazione (`ordinaButton`) viene configurato per aprire un dialog di ritiro se il carrello non è vuoto. In caso contrario, viene mostrato un messaggio Toast che informa

l'utente che il carrello è vuoto. Se il carrello contiene articoli, viene mostrato un dialog di ritiro, che imposta un listener per gestire l'evento del pulsante positivo del dialog.

onDialogPositiveClick(option: String, details: String, nome: String, numero: String):

vengono ottenute varie informazioni necessarie per creare un nuovo ordine, come l'ID utente, l'orario del ritiro, il tavolo, la data, il nome e il numero di telefono. Si ottiene la lista aggiornata del carrello e viene calcolato il totale dell'ordine. Una descrizione dell'ordine viene creata concatenando i dettagli degli articoli del carrello. Un nuovo ordine viene quindi aggiunto alla collezione "ordini" su Firebase, e in caso di successo, viene mostrato un messaggio Snackbar e il totale dell'ordine viene reimpostato a zero. In caso di errore, viene mostrato un messaggio Toast. Infine, la lista del carrello nell'adapter viene svuotata.

FragmentOrdini

```
private lateinit var recyclerView: RecyclerView
private lateinit var adapterOrdini: AdapterOrdini
private lateinit var ordiniList: MutableList<ItemOrdine>
private lateinit var auth: FirebaseAuth
private val db = FirebaseFirestore.getInstance()
private var role: String = ""
private var selectedButton: Button? = null
private lateinit var buttonAlTavolo: Button
private lateinit var buttonDAsporto: Button
private lateinit var progressBar: ProgressBar
```

onCreateView: Questa funzione esegue l'inflate del layout del fragment_ordini quando il fragment viene creato.

onViewCreated: vengono configurati e gestiti gli elementi dell'interfaccia utente in base al ruolo dell'utente. Vengono inizializzati i pulsanti di filtro, gestita la visibilità degli elementi per il personale, e impostati i listener per i pulsanti di filtro degli ordini. La RecyclerView viene configurata con un layout manager e un adattatore iniziale per visualizzare gli ordini.

selectButton: Questa funzione gestisce lo stato visivo dei pulsanti di filtro.

loadOrdini: Questa funzione recupera gli ordini da Firestore in base al ruolo dell'utente. Gli staff visualizzano tutti gli ordini, mentre i clienti vedono solo i propri. Utilizza un SnapshotListener per aggiornare la lista degli ordini in tempo reale e visualizzarli nel RecyclerView.

filterOrdini: Questa funzione filtra gli ordini in base al tipo e alla data di creazione. Poi aggiorna la lista degli ordini e aggiorna il RecyclerView.

onDialogPositiveClick: Questa funzione è un callback per quando viene eseguita un'azione positiva nel FragmentGestioneOrdine. Aggiorna lo stato e l'ora dell'ordine in Firestore e quindi aggiorna la lista degli ordini nel fragment.

updateOrderInList: Questa funzione aggiorna l'ordine nella lista locale in base alle modifiche apportate e aggiorna il RecyclerView.

createAdapter: Questa funzione crea e restituisce un'istanza di AdapterOrdini, controllando se l'utente loggato è staff.

FragmentStatistiche

```
private lateinit var db: FirebaseFirestore
private lateinit var radioGroupStatistiche: RadioGroup
private lateinit var barChart: BarChart
```

onCreateView (inflater: LayoutInflater, container: ViewGroup?, savedInstanceState: Bundle?): Gestisce l'inflate del layout del fragment, inizializza i componenti dell'interfaccia utente come il gruppo di radio buttons e il grafico a barre, e imposta un listener per i radio buttons per gestire il cambio di intervallo temporale selezionato dall'utente.

fetchOrders (timeInterval: String): Recupera gli ordini da Firebase in base all'intervallo temporale selezionato, calcola la quantità venduta di ciascun prodotto e aggiorna una map. I dati sono poi ordinati per quantità decrescente e passati alla funzione displayBarChart per visualizzarli nel grafico a barre.

displayBarChart (data: Map<String, Int>): Prepara i dati per il grafico a barre utilizzando le quantità vendute calcolate. Poi configura le varie impostazioni grafiche.

DettagliProdottoActivity

```
private val carrelloViewModel: CarrelloViewModel by viewModels()
private val RESULT_CODE_CARRELLO = 200
private val RESULT_CODE_PRODOTTO_ELIMINATO = 201
private var listaCarrello = arrayListOf<ItemCarrello>()
private lateinit var auth: FirebaseAuth
val db = Firebase.firestore
lateinit var role: String
lateinit var controllopreferito: Task<QuerySnapshot>
private var isConnected: Boolean = false
private lateinit var connectivityManager: ConnectivityManager
private lateinit var networkCallback: ConnectivityManager.NetworkCallback
private lateinit var progressBar: ProgressBar
private lateinit var progressBarFoto: ProgressBar
private lateinit var layoutDettagli: ConstraintLayout
private var deleteConfirmationDialog: AlertDialog? = null
```

onCreate (savedInstanceState: Bundle?): Questo metodo inizializza l'interfaccia utente leggendo il layout e impostando i vari componenti come il pulsante indietro. Gestisce anche la connettività di rete, registrando un callback per monitorare lo stato della connessione.

CheckInternetConnection(): verifica se il dispositivo è connesso a Internet. Se c'è una connessione, configura e visualizza i dettagli di un prodotto recuperato dall'Intent. In base al ruolo dell'utente modifica l'interfaccia utente, permettendo azioni come modificare o eliminare il prodotto. Scarica e mostra l'immagine del prodotto. Gestisce l'aumento o la diminuzione della quantità del prodotto e l'aggiunta al carrello. Implementa il tasto "preferiti". Se non c'è connessione, reindirizza l'utente alla MainActivity e chiude l'activity corrente.

RegisterNetworkCallback(): Registra una callback per rilevare cambiamenti nella connettività di rete. Quando la connessione è disponibile o persa, aggiorna dinamicamente l'interfaccia utente.

addToCart (itemName: String, foto: String?, prezzo: Double, quantity: Int): Gestisce l'aggiunta di un prodotto al carrello. Se il prodotto è già presente nel carrello, ne aumenta la quantità; altrimenti, aggiunge il prodotto con la quantità specificata. Aggiorna anche il ViewModel che gestisce l'elenco dei prodotti nel carrello.

addToFavorites (item: Item?): Aggiunge un prodotto ai preferiti dell'utente nel database Firebase, verificando prima se il prodotto è già tra i preferiti per evitare duplicazioni nel database.

eliminaProdotto (nomeItem: String, tipo: String): Elimina un prodotto specifico in Firebase tramite il nome passato alla funzione. Poi aggiorna l'interfaccia utente di conseguenza.

removeFromFavorites (item: Item?): Rimuove un prodotto dai preferiti dell'utente nel database Firebase, controllando prima che esista.

onProdottoModificato (item: Item): Questo metodo è chiamato quando un prodotto è stato modificato. Aggiorna l'interfaccia utente con i nuovi dettagli del prodotto e mostra un messaggio di notifica per informare l'utente dell'avvenuta modifica.

OnBackPressed(): Gestisce il comportamento quando l'utente preme il pulsante indietro del dispositivo. Restituisce i dati aggiornati del carrello all'activity chiamante usando un intent.

ReturnResult(): Restituisce i dati del carrello all'activity chiamante come risultato, aggiornando l'elenco dei prodotti nel carrello.

setupCollectionListener (tipo: String): Configura un listener per monitorare le modifiche ai prodotti in Firebase. Quando vengono rilevate modifiche, l'activity viene aggiornata dinamicamente.

FragmentGestioneOrdine

setListener (listener: GestioneOrdineListener): Imposta il listener per gestire le modifiche dell'ordine.

onCreate (savedInstanceState: Bundle?): Metodo chiamato durante la creazione del fragment. Recupera l'oggetto ordine passato come argomento tramite un bundle, se disponibile.

onCreateDialog (savedInstanceState: Bundle?): Crea e restituisce un dialog personalizzato per gestire l'ordine, accettandolo o rifiutandolo, con la possibilità di modificare l'ora di ritiro.

OnResume(): Metodo chiamato quando il fragment diventa visibile all'utente. Registra un BroadcastReceiver per monitorare lo stato della connessione Internet.

OnPause(): Metodo chiamato quando il fragment non è più visibile. Deregistra il BroadcastReceiver per evitare perdite di memoria.

RegisterNetworkReceiver(): Registra un BroadcastReceiver per rilevare cambiamenti nello stato della connettività di rete. Mostra un messaggio Toast se la connessione Internet viene persa durante l'interazione con il dialog.

UnregisterNetworkReceiver(): Deregistra il BroadcastReceiver quando il fragment non è più attivo, per evitare perdite di memoria.

ShowTimePickerDialog(): Mostra un dialog per selezionare un orario di ritiro dell'ordine con un controllo sull'intervallo di tempo.

FragmentModificaProdotto

setModificaProdottoListener (listener: ModificaProdottoListener): Imposta il listener ModificaProdottoListener per gestire le azioni di modifica del prodotto.

onCreateDialog (savedInstanceState: Bundle?): Apre un dialog per la modifica del prodotto. Gestisce anche il caricamento dell'immagine su Firebase Storage e l'aggiornamento dei dati del prodotto su Firestore.

onDismiss (dialog: DialogInterface): Metodo chiamato quando il dialog viene chiuso. Annulla il task di upload dell'immagine, se presente, per evitare operazioni inutili.

onActivityResult (requestCode: Int, resultCode: Int, data: Intent?): Metodo chiamato quando viene selezionata un'immagine dalla galleria. Carica l'immagine selezionata su Firebase Storage, aggiornando l'anteprima dell'immagine nel dialog.

showCurrentImage (item: Item): Mostra l'immagine attualmente associata al prodotto, se presente. In caso di errore nel caricamento, viene mostrata un'immagine di default.

showImagePreview (uri: Uri?): Mostra un'anteprima dell'immagine selezionata dall'utente nella ImageView del dialog.

uploadFoto (uri: Uri?, nomeFileFoto: String?): Carica l'immagine selezionata su Firebase Storage. Gestisce il progresso con una ProgressBar e aggiorna il controllo isImageUploaded in base all'esito dell'upload.

getFileName (uri: Uri?): Ottiene il nome del file dell'immagine selezionata dall'URI restituito dall'intent di selezione immagine.

OnResume(): Metodo chiamato quando il fragment diventa visibile all'utente. Registra un BroadcastReceiver per monitorare lo stato della connessione Internet.

OnPause(): Metodo chiamato quando il fragment non è più visibile. Deregistra il BroadcastReceiver per evitare perdite di memoria.

RegisterNetworkReceiver(): Registra un BroadcastReceiver per rilevare cambiamenti nello stato della connettività di rete. Mostra un messaggio Toast se la connessione Internet viene persa durante l'interazione con il dialog.

UnregisterNetworkReceiver(): Deregistra il BroadcastReceiver quando il fragment non è più attivo, per evitare perdite di memoria.

FragmentNuovoProdotto

```
private lateinit var storageRef: StorageReference
private var nomeFileFoto: String? = null
private val db = FirebaseFirestore.getInstance()
private var listener: NuovoProdottoListener? = null
private lateinit var progressBar: ProgressBar
private var isImageUploaded = false
private lateinit var imagePreview: ImageView
private var networkReceiver: BroadcastReceiver? = null
private var isInternetConnected: Boolean = true
private var uploadTask: UploadTask? = null
```

setNuovoProdottoListener(listener: NuovoProdottoListener): Questa funzione imposta un listener per gestire l'aggiunta di un nuovo prodotto. Il listener viene chiamato quando un nuovo prodotto viene aggiunto con successo.

onCreateDialog (savedInstanceState: Bundle?): Questa funzione apre un dialog per l'aggiunta di un nuovo prodotto. Gestisce l'interfaccia utente per l'inserimento dei dati del prodotto, inclusa l'upload dell'immagine su Firebase Storage e l'aggiornamento dei dati su Firestore. Mostra anche un'anteprima dell'immagine selezionata e gestisce l'evento di click sui pulsanti "Aggiungi" e "Annulla".

onDismiss (dialog: DialogInterface): Metodo chiamato quando il dialog viene chiuso. Annulla il task di upload dell'immagine, se presente, per evitare operazioni inutili.

onActivityResult (requestCode: Int, resultCode: Int, data: Intent?): Metodo chiamato quando viene selezionata un'immagine dalla galleria. Carica l'immagine selezionata su Firebase Storage, aggiornando l'anteprima dell'immagine nel dialog.

showImagePreview (uri: Uri?): Mostra l'immagine associata al prodotto, poi, se l'utente ne carica un'altra mostra un'anteprima dell'immagine selezionata dall'utente.

uploadFoto (uri: Uri?, nomeFileFoto: String?): Carica l'immagine selezionata su Firebase Storage. Gestisce il progresso con una ProgressBar e aggiorna il controllo isImageUploaded in base all'esito dell'upload.

getFileName (uri: Uri?): Ottiene il nome del file dell'immagine selezionata dall'URI restituito dall'intent di selezione immagine.

OnResume(): Metodo chiamato quando il fragment diventa visibile all'utente. Registra un BroadcastReceiver per monitorare lo stato della connessione Internet.

OnPause(): Metodo chiamato quando il fragment non è più visibile. Deregistra il BroadcastReceiver per evitare perdite di memoria.

RegisterNetworkReceiver(): Registra un BroadcastReceiver per rilevare cambiamenti nello stato della connettività di rete. Mostra un messaggio Toast se la connessione Internet viene persa durante l'interazione con il dialog.

UnregisterNetworkReceiver(): Deregistra il BroadcastReceiver quando il fragment non è più attivo, per evitare perdite di memoria.

FragmentRitiroDialog

```
private var listener: RitiroDialogListener? = null
private lateinit var editText: EditText
private var networkReceiver: BroadcastReceiver? = null
private var isConnected: Boolean = true
private var timePickerDialog: TimePickerDialog? = null
```

setListener(listener: RitiroDialogListener): Questa funzione consente di impostare un listener per gestire gli eventi del dialog di ritiro. Quando viene chiamata, il listener passato come parametro viene assegnato alla variabile di istanza listener.

onCreateDialog(savedInstanceState: Bundle?): Questa funzione apre il dialog di ritiro. Durante l'esecuzione, fa l'inflate del layout del dialog e recupera i riferimenti agli elementi UI necessari. Gestisce il cambio di stato nel RadioGroup per adattare dinamicamente l'interfaccia utente. Gestisce anche gli eventi dei pulsanti e l'input utente per validare e inviare i dati inseriti al listener appropriato.

showTimePickerDialog(): Questa funzione mostra un dialog di selezione dell'ora con un controllo sull'orario selezionato.

OnResume(): Metodo chiamato quando il fragment diventa visibile all'utente. Registra un BroadcastReceiver per monitorare lo stato della connessione Internet.

OnPause(): Metodo chiamato quando il fragment non è più visibile. Deregistra il BroadcastReceiver per evitare perdite di memoria.

RegisterNetworkReceiver(): Registra un BroadcastReceiver per rilevare cambiamenti nello stato della connettività di rete. Mostra un messaggio Toast se la connessione Internet viene persa durante l'interazione con il dialog.

UnregisterNetworkReceiver(): Deregistra il BroadcastReceiver quando il fragment non è più attivo, per evitare perdite di memoria.

AdapterCarrello

```
auth = Firebase.auth
val authid = (auth.currentUser?.uid).toString()
val ordiniCollection = db.collection("ordini")
val oraRitiro = if (option == "Servizio d'Asporto") details else ""
val tavolo = if (option == "Servizio al Tavolo") details else ""
val formatter = DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss")
val data = LocalDateTime.now().format(formatter).toString()
val nome = if (option == "Servizio al Tavolo") "" else nome
val numero = if (option == "Servizio al Tavolo") "" else numero
```

onCreateViewHolder(parent: ViewGroup, viewType: Int): Crea e restituisce un nuovo CarrelloViewHolder eseguendo l'inflate del layout R.layout.item_carrello.

onBindViewHolder(holder: CarrelloViewHolder, position: Int): Associa i dati dell'elemento nella posizione specificata dalla lista listaCarrello alla vista rappresentata dal CarrelloViewHolder. Gestisce il caricamento dell'immagine del prodotto.

calcolaTotale(): Calcola e restituisce il totale del prezzo del carrello basato sulla somma delle quantità moltiplicate per il prezzo di ciascun elemento nella lista listaCarrello.

getItemCount(): Restituisce il numero totale di elementi nella lista listaCarrello.

setData(listaCarrello: MutableList<ItemCarrello>): Imposta i dati del carrello con una nuova lista listaCarrello e notifica al RecyclerView di aggiornare la visualizzazione.

getList(callback: (List<ItemCarrello>) -> Unit): Restituisce la lista corrente listaCarrello attraverso una callback fornita come parametro.

clearList(): Rimuove tutti gli elementi dalla lista listaCarrello e notifica al RecyclerView di aggiornare la visualizzazione.

AdapterListeHome

onCreateViewHolder(parent: ViewGroup, viewType: Int): Questa funzione crea e restituisce un nuovo MyViewHolder eseguendo l'inflate del layout R.layout.each_item utilizzato per visualizzare ciascun elemento della lista.

getItemCount(): Questa funzione restituisce il numero totale di elementi nella lista listaProdotti.

onBindViewHolder(holder: MyViewHolder, position: Int): Questa funzione associa i dati dell'elemento nella posizione specificata dalla lista listaProdotti alla vista rappresentata dal MyViewHolder. Gestisce anche il caricamento dell'immagine del prodotto. Se l'elemento ha una foto associata, carica l'immagine dallo storage Firebase; altrimenti, imposta un'immagine di default.

setFilteredList(listaProdotti: List<Item>): Questa funzione imposta la lista di prodotti con una nuova lista filtrata listaProdotti e notifica al RecyclerView di aggiornare la visualizzazione dei dati.

updateList(newList: List<Item>): Questa funzione aggiorna completamente la lista di prodotti con una nuova lista newList e notifica al RecyclerView di aggiornare la visualizzazione dei dati.

AdapterOrdini

onCreateViewHolder(parent: ViewGroup, viewType: Int): Questa funzione crea e restituisce un nuovo OrdiniViewHolder facendo l'inflate del layout R.layout.item_ordine.

getItemCount(): Questa funzione restituisce il numero totale di elementi nella lista ordiniList.

onBindViewHolder(holder: OrdiniViewHolder, position: Int): Questa funzione associa i dati dell'ordine nella posizione specificata dalla lista ordiniList alla vista rappresentata dal OrdiniViewHolder. Gestisce la visualizzazione delle informazioni dell'ordine filtrandole in base al ruolo dell'utente loggato.

formatOrderDescription(orderDescription: String): Questo metodo serve a modificare come viene visualizzata la descrizione dell'ordine nella vista "ordini".

AdapterStatistiche

onCreateViewHolder(parent: ViewGroup, viewType: Int): Questa funzione viene chiamata quando il RecyclerView ha bisogno di creare un nuovo ViewHolder per rappresentare un elemento della lista. In questo caso, viene fatto l'inflate del layout R.layout.item_statistica e viene creato un nuovo oggetto ViewHolder di tipo ViewHolder associato a questo layout.

onBindViewHolder(holder: ViewHolder, position: Int): Questa funzione popola i dati dell'elemento nella posizione specificata all'interno del RecyclerView. Prende il nome del prodotto e il conteggio dal productCount e li imposta nei TextView corrispondenti all'interno del ViewHolder.

getItemCount(): Questa funzione restituisce il numero totale di elementi nella lista productCount.

(classe interna) ViewHolder(itemView: View): definisce la struttura di ogni elemento nella vista del RecyclerView. Contiene riferimenti ai TextView productName e productCount che verranno popolati con i dati di ciascun elemento.

CarrelloViewModel

getItems(): Restituisce una copia immutabile della lista corrente degli elementi del carrello.

setItems(newItems: ArrayList<ItemCarrello>?): Imposta una nuova lista mutabile di elementi del carrello con quella fornita. Se newItems è nullo, imposta una lista vuota.

Item

writeToParcel(parcel: Parcel, flags: Int): Questa funzione scrive i dati dell'oggetto corrente nel Parcel fornito.

describeContents(): Questa funzione restituisce un intero che rappresenta un valore flag, che può essere utilizzato per descrivere il tipo di dati contenuti nel Parcelable.

createFromParcel(parcel: Parcel): legge i dati da un oggetto Parcel e crea una nuova istanza della classe Item utilizzando quei dati

newArray(size: Int): Questa funzione crea e restituisce un array contenente elementi di tipo Item, inizializzati tutti a null.

ItemCarrello

writeToParcel(parcel: Parcel, flags: Int): Questa funzione scrive i dati dell'oggetto corrente nel Parcel fornito.

describeContents(): Questa funzione restituisce un intero che rappresenta un valore flag, che può essere utilizzato per descrivere il tipo di dati contenuti nel Parcelable.

createFromParcel(parcel: Parcel): legge i dati da un oggetto Parcel e crea una nuova istanza della classe Item utilizzando quei dati

newArray(size: Int): Questa funzione crea e restituisce un array contenente elementi di tipo Item, inizializzati tutti a null.

ItemOrdine

writeToParcel(parcel: Parcel, flags: Int): Questa funzione scrive i dati dell'oggetto corrente nel Parcel fornito.

describeContents(): Questa funzione restituisce un intero che rappresenta un valore flag, che può essere utilizzato per descrivere il tipo di dati contenuti nel Parcelable.

createFromParcel(parcel: Parcel): legge i dati da un oggetto Parcel e crea una nuova istanza della classe Item utilizzando quei dati

newArray(size: Int): Questa funzione crea e restituisce un array contenente elementi di tipo Item, inizializzati tutti a null.

2.7.2 Descrizione Componenti (Flutter)

Main

main() Inizializza l'applicazione, imposta l'orientamento dello schermo su verticale e avvia l'app con ChangeNotifierProvider per gestire lo stato globale.

Login

initState () Viene chiamato `_checkAutoLogin` per verificare le credenziali salvate e procedere con il login automatico se presenti, inoltre monitora i cambiamenti nello stato della connessione a Internet.

dispose() L'applicazione interrompe l'ascolto dei cambiamenti nello stato della connessione a Internet

_checkAutoLogin() Recupera le credenziali salvate e se sono presenti, popola i controller e chiama `_login`.

_login() Mostra un indicatore di caricamento, autentica l'utente usando email e password, se il login ha successo naviga alla MainPage, gestisce eventuali errori di autenticazione mostrando dei messaggi.

build() Verifica lo stato della connessione Internet e mostra la LoginScaffold se è connesso, altrimenti mostra il NoInternetScaffold

Register

initState () Metodo che viene chiamato quando il widget viene creato per la prima volta, imposta un listener e monitora la connessione a internet, se la connessione cambia anche l'interfaccia si aggiorna.

dispose() Quando il widget viene rimosso dalla schermata questo metodo annulla il listener per la connessione a internet.

_register() Quando l'utente preme il pulsante di registrazione viene chiamato questo metodo che prova a creare un nuovo account, se ci riesce, mostra un messaggio e naviga alla pagina di login, in caso negativo mostra un messaggio di errore.

MainPage

initState () Quando il widget viene creato inizializza le istanze 'auth' e 'db' di firebase, monitora lo stato della connessione a internet e chiama `_checkUserRole()` per determinare il ruolo dell'utente attualmente loggato.

dispose() Quando il widget viene rimosso dalla schermata questo metodo annulla il listener per la connessione a internet.

_checkUserRole() Metodo che recupera l'utente corrente da firebase authentication, e se l'utente esiste recupera il ruolo dal firestore, di conseguenza lo stato di 'role' viene aggiornato.

_onItemTapped(int index) Metodo che gestisce ogni tap sulle icone della bottomNavigationBar, ad ogni tocco aggiorna lo stato di `_selectedIndex` per cambiare la pagina visualizzata.

Home

dispose() Pulisce la memoria liberando le risorse utilizzate dal `_searchController`.

_logout() Gestisce l'operazione di logout dell'utente corrente, effettua il logout da firebase authentication utilizzando `FirebaseAuth.instance.signOut()`, rimuove le credenziali memorizzate localmente e dopo il logout naviga alla pagina di login

_clearSearchField() Metodo che svuota il campo di ricerca e aggiorna lo stato del widget per permettere di iniziare una nuova ricerca senza campo preesistente.

GenericItem

Build(BuildContext context) Metodo utilizzato per costruire l'interfaccia utente del widget. Mentre è in attesa dell'URL dell'immagine, che ottiene tramite `FutureBuilder` mostra un indicatore di caricamento, se c'è un errore mostra la relativa icona, altrimenti visualizza l'immagine dell'oggetto, il nome, la descrizione e il prezzo in una card.

GenericList

_showToast(BuildContext context, String message) Mostra un messaggio al centro dello schermo.

_getImageUrl(String imageName) Recupera l'URL dell'immagine dal Firebase Storage. Se c'è un errore nel recupero dell'URL restituisce un URL di default.

DettagliProdotto

_initializeData() Chiama i metodi **_checkRuolo** e **_checkDocInOfferte**.

_checkRuolo() Verifica se l'utente corrente ha il ruolo di staff leggendo il documento utente nel database firestore.

_checkDocInOfferte() Controlla se il prodotto corrente è presente nella collezione "offerte" di firestore.

_incrementaQuantita() Incrementa la quantità del contatore del prodotto.

_decrementaQuantita() Decrementa la quantità del contatore del prodotto evitando che scenda sotto lo zero.

_aggiornaQuantita() Aggiorna la quantità del prodotto direttamente con il valore aggiunto dall'utente nel contatore.

_showEliminaConfermaDialog(BuildContext context) Mostra un dialog di conferma per l'eliminazione del prodotto, inoltre verifica che ci sia connessione ad internet, in caso contrario chiude il dialog.

_eliminaProdotto() Elimina il prodotto dalla collezione "offerte" di firestore.

_showModificaOffertaDialog(BuildContext context) Mostra un dialog per modificare le informazioni dell'offerta.

_getImageUrl(String imageName) Recupera l'URL dell'immagine del prodotto dal Firebase Storage. Se c'è un errore nel recupero dell'URL restituisce un URL di default.

_onOfferEdited() Aggiorna le informazioni dell'offerta visualizzate nella schermata se vengono modificate nel database.

Offerte

_checkUserRole() Metodo che verifica il ruolo dell'utente corrente. Se l'utente è autenticato il metodo recupera il documento corrispondente all'utente dalla collezione users di firestore. Se il ruolo dell'utente è "staff", aggiorna lo stato di isStaff a true.

_fetchDataFromFirebase() Metodo che recupera tutte le offerte dalla relativa collezione di firestore, converte poi tutti i documenti recuperati in una lista di oggetti GenericItem, per poi aggiornare lo stato per riflettere la fine del caricamento.

_showNewOfferDialog() Questo metodo mostra un dialog per aggiungere una nuova offerta alla lista. Utilizza il widget NewOfferDialog e passa una callback da eseguire quando una nuova offerta viene aggiunta.

CarrelloModel

```
class CarrelloModel {  
    final String name;  
    final String image;  
    final String description;  
    final double price;  
    int quantity;  
  
    CarrelloModel({  
        this.name = "",  
        this.image = "",  
        this.description = "",  
        this.price = 0,  
        this.quantity = 0,  
    });  
}
```

CarrelloProvider

addToCarrello(CarrelloModel item, {int quantity = 1}) Questo metodo aggiunge un prodotto al carrello. Se il prodotto è già presente nella lista ne incrementa la quantità, altrimenti lo aggiunge come nuovo elemento. Dopo aver modificato il carrello, avvisa i widget in ascolto che lo stato è cambiato.

removeFromCarrello(CarrelloModel item) Rimuove un prodotto dal carrello. Se il prodotto è già presente ne decrementa la quantità. Se la quantità del prodotto scende sotto 1, lo rimuove dalla lista. Dopo aver modificato il carrello, avvisa i widget in ascolto che lo stato è cambiato.

calcolaTotale() Calcola il totale del carrello sommando il prezzo di ciascun prodotto moltiplicato per la sua quantità.

getCurrentUserID() Metodo che recupera l'UID dell'utente attualmente loggato utilizzando firebase authentication. Se l'utente è loggato stampa il suo UID, altrimenti stampa un messaggio che indica che nessun utente è loggato.

creaOrdine(OrdineModel nuovoOrdine) Questo metodo crea un nuovo ordine utilizzando le informazioni fornite e il contenuto del carrello.

Carrello

È un widget stateless che rappresenta la pagina del carrello nell'app. Utilizza il provider “CarrelloProvider” per gestire lo stato del carrello e mostrare i prodotti aggiunti. Costruisce dinamicamente la lista dei prodotti nel carrello e aggiorna automaticamente la UI quando lo stato cambia. Inoltre, permette all'utente di aggiungere o rimuovere prodotti e procedere con l'ordine tramite pulsanti che chiamano i dialog.

OrdineModel

```
class OrdineModel {  
  final String id;  
  final String data;  
  final String descrizione;  
  final String nome;  
  final String ora;  
  final String stato;  
  final String tavolo;  
  final String telefono;  
  final String tipo;  
  final String totale;
```

```
  OrdineModel({  
    this.id = "",  
    this.data = "",  
    this.descrizione = "",  
    this.nome = "",  
    this.ora = "",  
    this.stato = "",  
    this.tavolo = "",  
    this.telefono = "",  
    this.tipo = "",  
    this.totale = "",  
  });  
}
```

OrdineDialog

Questo è un widget stateful che mostra un dialog per gestire un ordine specifico. Permette di accettare o rifiutare l'ordine. Nel caso di un ordine da asporto consente di specificare un orario di ritiro.

Ordini

initState() Questo metodo viene chiamato una sola volta quando il widget viene inserito nell'albero widget, inizializza Firebase Authentication (auth) e Firestore (db) e chiama `_checkUserRole()` per determinare il ruolo dell'utente e caricare gli ordini di conseguenza.

_checkUserRole() Metodo che verifica il ruolo dell'utente corrente. Se l'utente è staff carica gli ordini degli ultimi 24 ore, se è un cliente carica i suoi ordini.

_loadOrdini(String userId) Carica gli ordini di un utente specifico dal Firestore ordinandoli per data in ordine decrescente.

_filterOrdini(String tipo) Filtra gli ordini per tipo "Servizio al Tavolo" o "Servizio d'Asporto" e restituisce solo quelli effettuati nelle ultime 24 ore.

_refreshOrdini() Ricarica la lista degli ordini in base al ruolo dell'utente e al tipo di ordine selezionato.

build(BuildContext context) Questo metodo costruisce l'interfaccia utente del widget, se l'utente è staff mostra i pulsanti per filtrare gli ordini e la lista degli ordini filtrati, se l'utente è un cliente mostra la lista dei suoi ordini.

CreaOrdineDialog

initState() Questo metodo si occupa di inizializzare il listener per il controllo della connessione internet utilizzando `InternetConnectionChecker`.

dispose() Quando il widget viene rimosso dalla schermata, il metodo `dispose` annulla il listener per la connessione a internet.

_resetForm() Viene chiamato quando si seleziona un nuovo tipo di servizio, cancella i valori dei controller dei campi di testo, serve quindi per reimpostare lo stato del form.

_buildCampiDinamici() Costruisce dinamicamente i campi di input basati sul tipo di servizio selezionato. Se viene selezionato Servizio al Tavolo, mostra un campo per inserire il numero del tavolo. Se viene selezionato Servizio d'asporto, mostra campi per l'ora di ritiro, il nome e il numero di telefono del cliente.

build(BuildContext context) Mostra due opzioni radio per selezionare il tipo di servizio e visualizza dinamicamente i campi di input appropriati in base alla selezione. Gestisce la validazione del form e la creazione di un nuovo ordine quando l'utente preme il pulsante "OK".

ModificaOffertaDialogState

initState () Esegue inizializzazioni come l'impostazione del listener per la connessione a internet e l'assegnazione dei valori iniziali ai controller dei campi di testo.

dispose() Quando il widget viene rimosso dalla schermata annulla il listener della connessione a internet.

_pickImage() Utilizza la libreria `image_picker` per selezionare un'immagine dalla galleria. Una volta selezionata un'immagine, aggiorna lo stato del widget con il file dell'immagine e avvia il caricamento dell'immagine chiamando `_uploadImage`.

_uploadImage() Questo metodo carica l'immagine selezionata su Firebase Storage. Imposta `_isUploading` a true durante il caricamento e aggiorna `_uploadedImageUrl` con l'URL dell'immagine caricata, comunica poi l'esito con un messaggio di conferma o di errore.

_updateOffer() Questo metodo aggiorna l'offerta su Firestore, se l'offerta è aggiornata con successo, chiude il dialog e richiama il callback `onOfferEdited`.

NewOfferDialogState

initState () Esegue inizializzazioni come l'impostazione del listener per la connessione a internet e l'assegnazione dei valori iniziali ai controller dei campi di testo.

dispose() Quando il widget viene rimosso dalla schermata annulla il listener della connessione a internet.

_pickImage() Utilizza la libreria `image_picker` per selezionare un'immagine dalla galleria. Una volta selezionata un'immagine, aggiorna lo stato del widget con il file dell'immagine e avvia il caricamento dell'immagine chiamando `_uploadImage`.

_uploadImage() Questo metodo carica l'immagine selezionata su Firebase Storage, aggiorna lo stato di caricamento e mostra un messaggio di successo o di errore.

_addOffer() Controlla se i campi del form sono validi e se un'immagine è stata caricata. Se un'offerta con lo stesso nome esiste già, mostra un messaggio di errore. Se tutto è a posto, aggiunge la nuova offerta nel database e chiude il dialog.

NoInternetScaffold

È un widget di tipo `stateless` che fornisce una semplice interfaccia utente per informare l'utente che non c'è connessione internet.

AppColors

```
import 'package:flutter/material.dart';  
class AppColors {  
  static const Color primaryColor = Color(0xFFFF9576A);  
  static const Color secondaryColor = Color(0xFFFD62);  
  static const Color lightYellow =Color.fromARGB(64, 253, 220, 2);  
  static const Color darkGrey= Color.fromARGB(150, 2, 2, 2);  
  static const Color myGrey = Color.fromARGB(255, 105, 107, 110);  
}
```

2.8 Testing

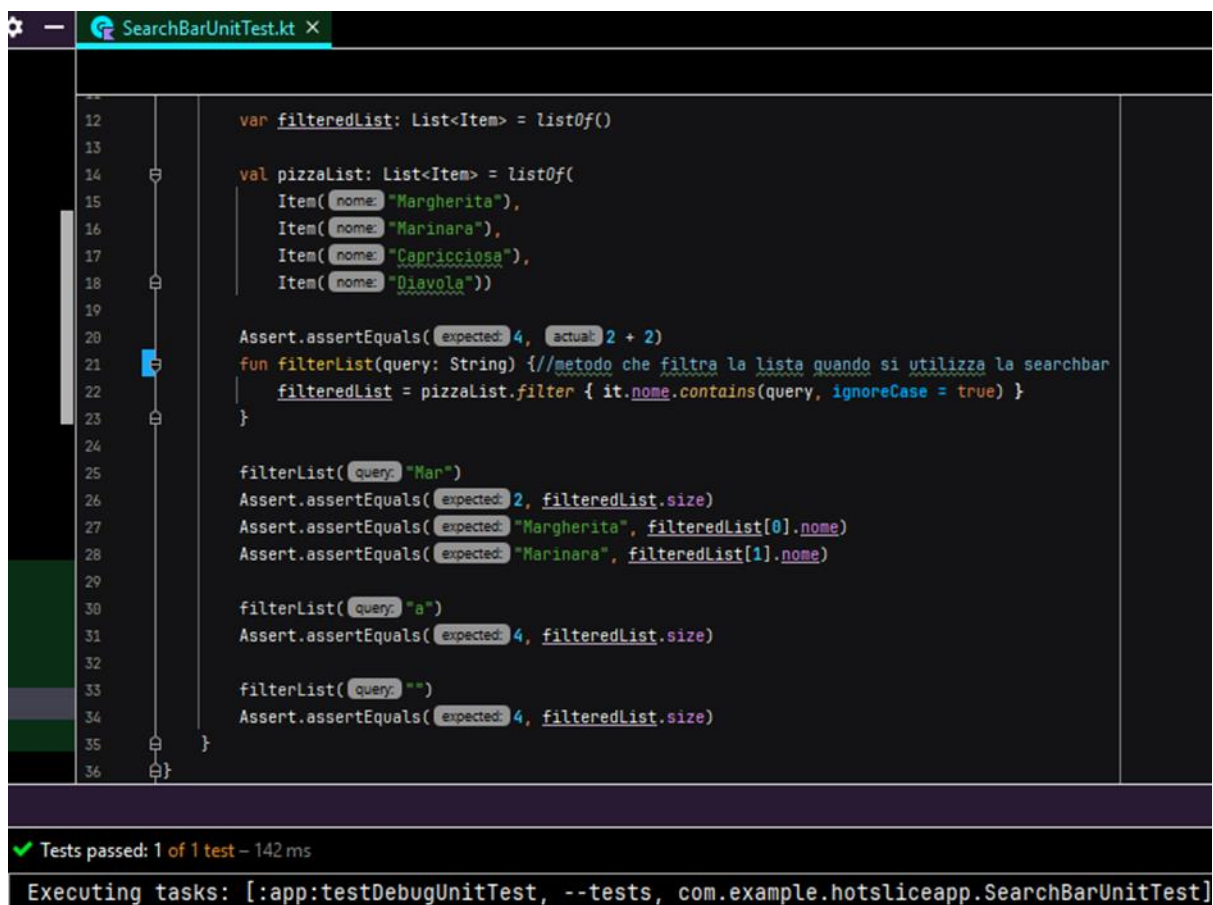
Abbiamo realizzato sia Unit test, volti a testare singole funzionalità, sia Instrumented test, che testano il comportamento di porzioni dell'applicazione simulando l'esecuzione e l'utilizzo da parte di un utente.

2.8.1 Unit test

Gli Unit test sono un metodo di test del codice, che consentono di testare singole unità di codice e verificare che i risultati ottenuti coincidano con i risultati previsti.

Nel nostro caso le unità testate sono metodi di activities e/o fragments o funzioni utilizzate al loro interno.

- SearchBarUnitTest



```
12 var filteredList: List<Item> = listOf()
13
14 val pizzaList: List<Item> = listOf(
15     Item( nome: "Margherita"),
16     Item( nome: "Marinara"),
17     Item( nome: "Capricciosa"),
18     Item( nome: "Diavola"))
19
20 Assert.assertEquals( expected: 4, actual: 2 + 2)
21 fun filterList(query: String) { //metodo che filtra la lista quando si utilizza la searchbar
22     filteredList = pizzaList.filter { it.nome.contains(query, ignoreCase = true) }
23 }
24
25 filterList( query: "Mar")
26 Assert.assertEquals( expected: 2, filteredList.size)
27 Assert.assertEquals( expected: "Margherita", filteredList[0].nome)
28 Assert.assertEquals( expected: "Marinara", filteredList[1].nome)
29
30 filterList( query: "a")
31 Assert.assertEquals( expected: 4, filteredList.size)
32
33 filterList( query: "")
34 Assert.assertEquals( expected: 4, filteredList.size)
35 }
36 }
```

✓ Tests passed: 1 of 1 test - 142 ms

Executing tasks: [:app:testDebugUnitTest, --tests, com.example.hotsliceapp.SearchBarUnitTest]

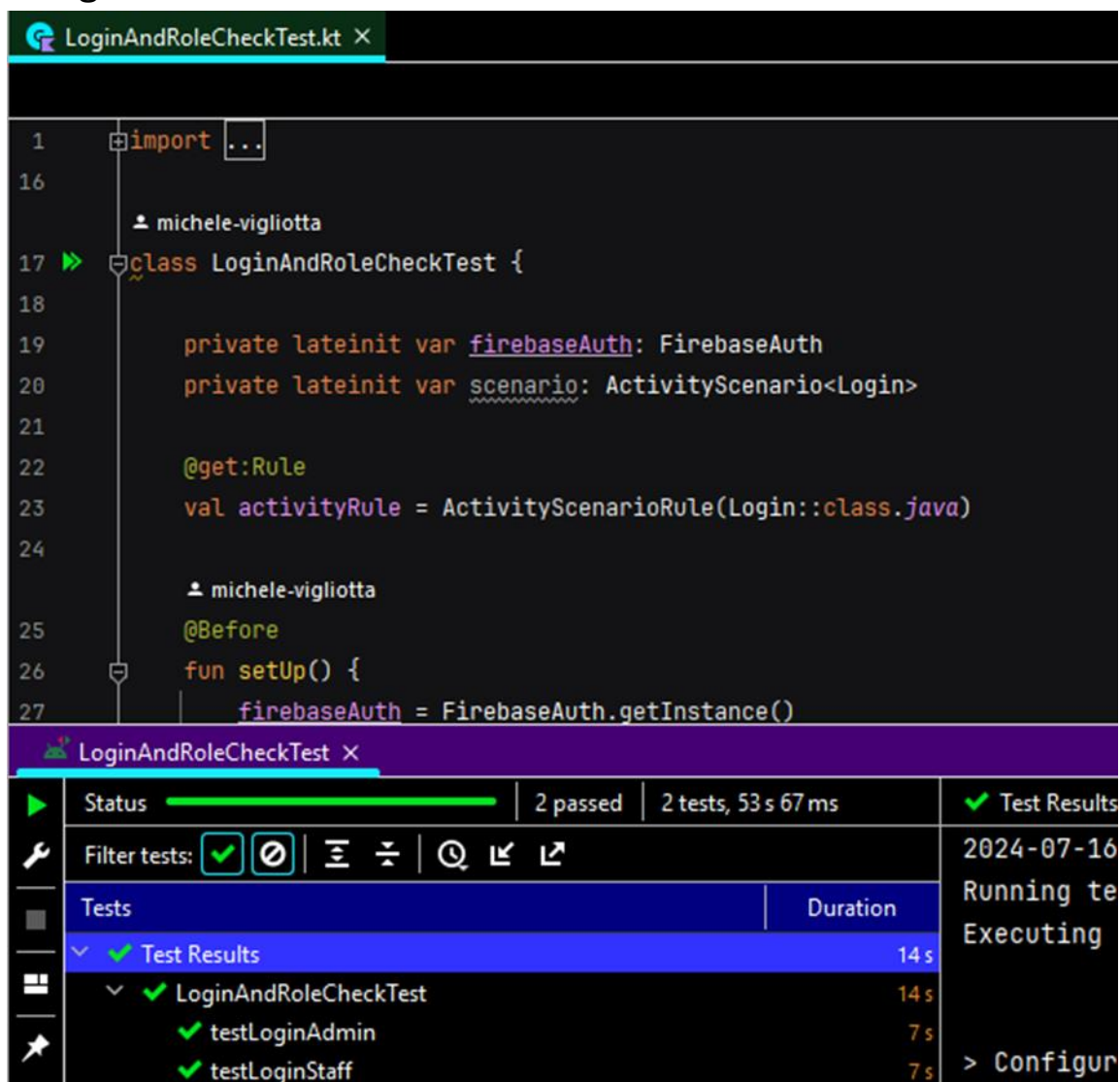
In questo test abbiamo verificato il corretto funzionamento del metodo *filterList()* utilizzato nel *FragmentHome* per gestire i risultati della ricerca in base alla stringa inserita dall'utente nella search bar. La funzione restituisce tutti gli Item(classe utilizzata per i prodotti) che contengono nel nome i caratteri inseriti nella search bar. In particolare, l'assert a riga 34(nella foto soprastante) verifica che vengano mostrati tutti gli Item nel caso in cui non venga inserito nessun carattere nella search bar. Di seguito, l'aspetto della search bar all'interno dell'app.

2.8.2 Instrumented test

Inoltre abbiamo eseguito degli Instrumented test. Si trattano di test che vengono eseguiti su un dispositivo o un emulatore Android. Lo scopo è quello di valutare il comportamento del codice in un contesto che simula le reali condizioni di esecuzione.

Per automatizzare l'interazione con l'interfaccia utente, abbiamo utilizzato il framework Espresso di Google. Questo ci ha permesso di testare la corretta navigazione dell'app e verificare la correttezza della logica assegnata a vari *button* o alla creazione di alcune *viste*.

- LoginAndRoleCheckTest



The screenshot displays the Android Studio IDE. The top pane shows the Kotlin code for `LoginAndRoleCheckTest.kt`. The code includes imports, class-level variables for `firebaseAuth` and `scenario`, an `@get:Rule` annotation for `activityRule`, and a `@Before` annotated `setUp()` method that initializes `firebaseAuth`.

The bottom pane shows the test results. A status bar indicates "2 passed | 2 tests, 53 s 67 ms". Below this, a table lists the tests:

Tests	Duration
✓ Test Results	14 s
✓ LoginAndRoleCheckTest	14 s
✓ testLoginAdmin	7 s
✓ testLoginStaff	7 s

On the right side of the results pane, the date "2024-07-16" and the status "Running te Executing" are visible, along with a "> Configur" link.

In questo test abbiamo verificato il corretto funzionamento del login e la modifica dinamica della *bottom navigation bar* in base al ruolo dell'utente attualmente loggato. Nell'annotazione `@get:Rule` specifichiamo l'activity(Login, in questo caso) da testare utilizzando l' `ActivityScenarioRule` che è una classe che consente di gestire il ciclo di vita di un'activity.

Nella funzione `setUp()`, che si occupa della configurazione dell'ambiente prima di

eseguire i test, istanziamo FirebaseAuth ed effettuiamo il logout.

Nella funzione *tearDown()*, che esegue operazioni di pulizia dell'ambiente dopo l'esecuzione dei test, effettuiamo il logout. In questo modo abbiamo la certezza di non aver nessun utente loggato prima e dopo ogni funzione di test.

In questa classe, abbiamo due funzioni di test: *testLoginStaff()* e *testLoginAdmin()*.

In *testLoginStaff()* effettuiamo il login con l'utente staff@gmail.com che ha ruolo *staff* in Firestore.

```
role: "staff"
```

Dopo il login, verifichiamo attraverso delle asserzioni che la *bottom navigation bar* sia visibile e che le icone visualizzate siano appropriate per un utente con ruolo *staff*.

In *testLoginAdmin()* effettuiamo il login con l'utente admin@gmail.com che ha ruolo *admin* in Firestore.

```
onView(withId(R.id.bottomNavigationView)).check(ViewAssertions.matches(isDisplayed()))
onView(withId(R.id.bottom_offerte)).check(ViewAssertions.matches(isDisplayed()))
onView(withId(R.id.bottom_home)).check(ViewAssertions.matches(isDisplayed()))
onView(withId(R.id.bottom_ordini)).check(ViewAssertions.matches(isDisplayed()))
```

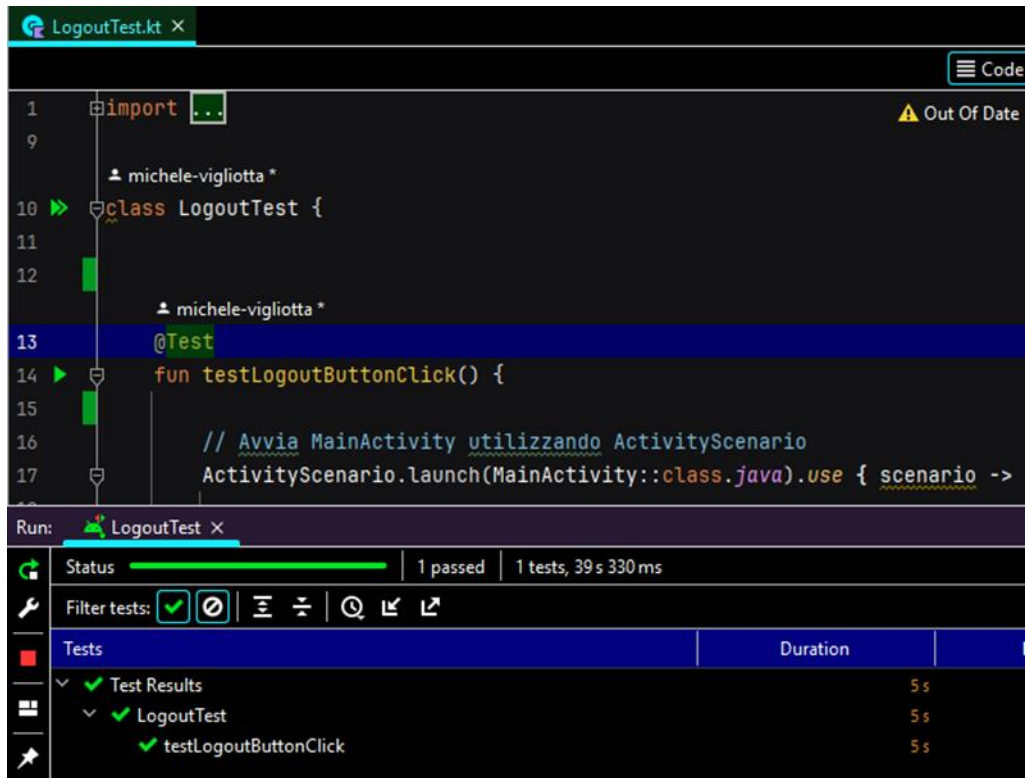
In *testLoginAdmin()* effettuiamo il login con l'utente admin@gmail.com che ha ruolo *admin* in Firestore.

```
role: "admin"
```

Dopo il login, verifichiamo attraverso delle asserzioni che la *bottom navigation bar* sia visibile e che le icone visualizzate siano appropriate per un utente con ruolo *staff*.

```
onView(withId(R.id.bottomNavigationView)).check(ViewAssertions.matches(isDisplayed()))
onView(withId(R.id.bottom_home)).check(ViewAssertions.matches(isDisplayed()))
onView(withId(R.id.bottom_statistiche)).check(ViewAssertions.matches(isDisplayed()))
```

- LogoutTest



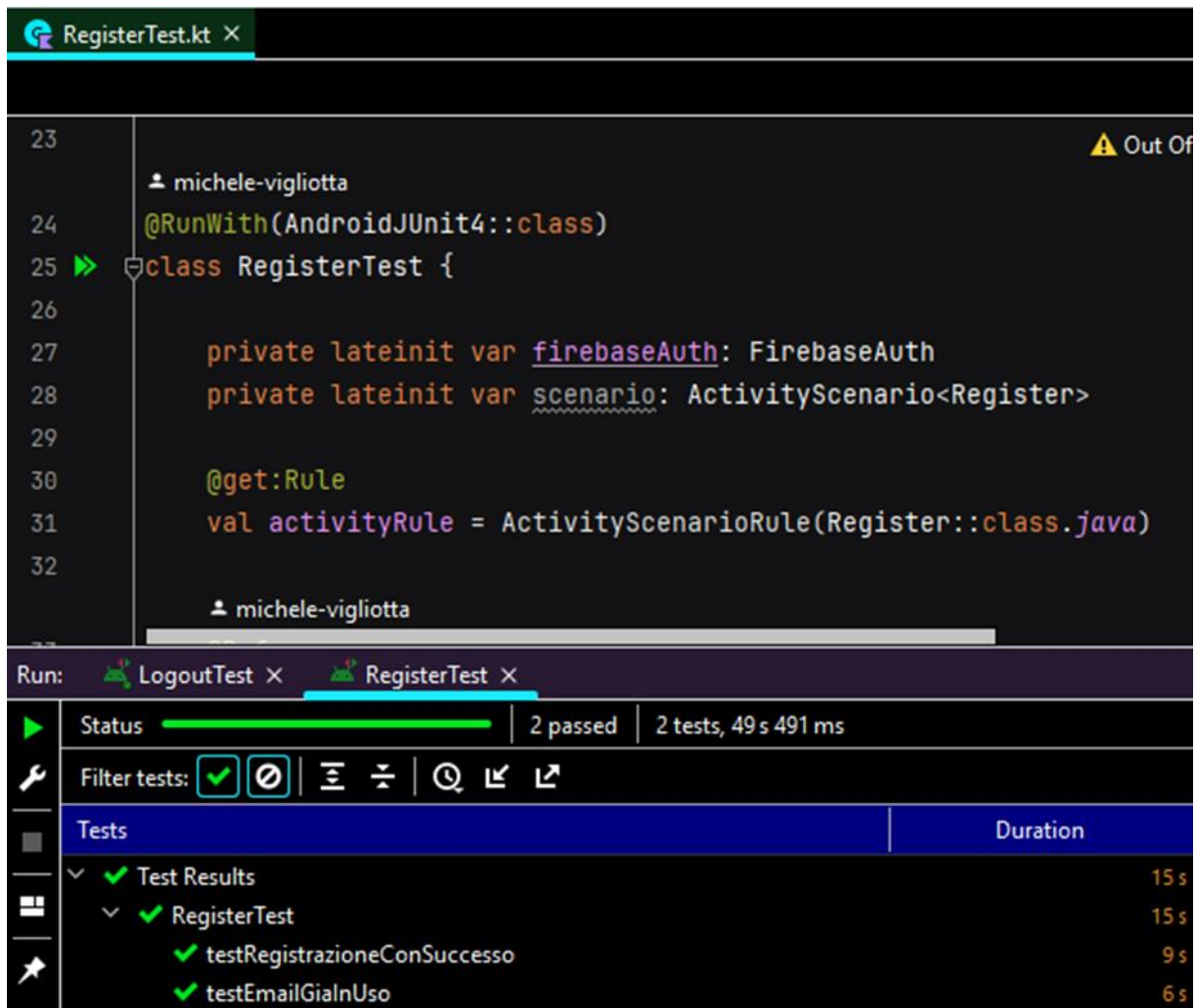
In questo test abbiamo verificato il corretto funzionamento del logout.

In particolare abbiamo utilizzato un'asserzione per accertarci che l'utente venga reindirizzato alla vista *Login* dopo aver effettuato il logout.

```
onView(withId(R.id.loginActivityRoot))  
    .check(matches(isDisplayed())) ^use
```

`loginActivityRoot` è l'id del layout principale della nostra activity *Login*.

- RegisterTest



```

23
24  @RunWith(AndroidJUnit4::class)
25  class RegisterTest {
26
27      private lateinit var firebaseAuth: FirebaseAuth
28      private lateinit var scenario: ActivityScenario<Register>
29
30      @get:Rule
31      val activityRule = ActivityScenarioRule(Register::class.java)
32
  
```

Run: LogoutTest x RegisterTest x

Status 2 passed | 2 tests, 49 s 491 ms

Filter tests: ☒ ☐ ☐ ☐ ☐ ☐ ☐

Tests	Duration
✓ Test Results	15 s
✓ RegisterTest	15 s
✓ testRegistrazioneConSuccesso	9 s
✓ testEmailGiaInUso	6 s

In questo test abbiamo verificato il corretto funzionamento della registrazione dell'utente e della corretta navigazione in seguito. Nell'annotazione `@get:Rule` specifichiamo l'activity(Register, in questo caso) da testare utilizzando la classe `ActivityScenarioRule`. Nella funzione `setUp()` instanziamo `FirebaseAuth`. Nella funzione `tearDown()` effettuiamo il logout. In questo modo siamo sicuri di non aver nessun utente loggato dopo ogni funzione di test.

In questa classe, abbiamo due funzioni di test: `testRegistrazioneConSuccesso()` e `testEmailGiaInUso()`.

In `testRegistrazioneConSuccesso()` registriamo un utente di prova su Firebase Authentication e verifichiamo che la registrazione sia andata a buon fine controllando che l'utente sia reindirizzato alla vista del login.

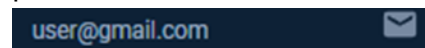
```

onView(withId(R.id.loginActivityRoot))
    .check(matches(isDisplayed()))
  
```

Alla fine eliminiamo l'utente appena registrato per non compromettere la prossima esecuzione della stessa funzione di test.

In `testEmailGiaInUso()` andiamo a registrare un utente già registrato e verifichiamo che

la registrazione non avvenga. L'email utilizzata è user@gmail.com che risulta già presente in Firebase Authentication.



La verifica che la registrazione non avvenga viene fatta con un'asserzione che confronta l'eccezione che otteniamo da FirebaseAuth con l'eccezione FirebaseAuthUserCollisionException che viene restituita quando un utente è già registrato.

3. Conclusioni

Lo sviluppo di questa applicazione in Kotlin e in Flutter è stata un'esperienza molto formativa e stimolante. Per noi è stato il primo sguardo sulla programmazione mobile ed è stato un ottimo modo per iniziare a conoscere in modo più approfondito ciò che utilizziamo quotidianamente.

Desideriamo esprimere un sentito ringraziamento al Professore *Emanuele Storti* che ci ha illustrato le tecniche moderne di programmazione mobile e permettendoci di avvicinarci di più a questo mondo.

Un sincero ringraziamento va a tutto il team per la collaborazione, l'impegno e la dedizione dimostrati nel corso del progetto.

Un ringraziamento anche ai nostri amici e parenti che ci hanno fornito degli ottimi consigli per rendere l'applicazione più piacevole all'uso e alla vista.